arXiv:2601.19239v2 [cs.SE] 25 Sep 2026

LLM-based Vulnerability Detection at Project Scale: An Empirical Study

Fengjie Li, Jiajun Jiang, Dongchi Chen, Yingfei Xiong

***Abstract*—As software complexity grows, security vulnerabilities become more frequent and harmful, making automated vulnerability detection essential. While recent LLM-based detectors combine semantic reasoning with static analysis for project-scale scanning, their real-world effectiveness remains unclear, and the underlying root causes of their failures are still underexplored. In this paper, we present the first comprehensive empirical study of specialized LLM-based detectors and compare them with traditional static analyzers at the project scale. Specifically, our study evaluates five of the latest and representative specialized LLM-based methods, two general Agent-based methods and four traditional tools using: 1) a controlled benchmark of 265 known real-world vulnerabilities (C/C++ and Java) to assess detection capability, and 2) 24 active open-source projects/modules, where we analyzed 6,442 sampled warnings using Codex-assisted labeling with stratified manual validation and classified the root causes of 5,896 false positives using a taxonomy derived from 355 manually inspected reports.**

Our evaluation yields three key findings: First, general agent methods achieve the highest recall, but still struggle with complex code. And both LLM-based and traditional methods suffer from incomplete API modeling. Second, in open-source projects/modules, both LLM-based and traditional tools generate substantial warnings but suffer from very high false discovery rates, hindering practical use. Our analysis further reveals truncated interprocedural context and incorrect program-point classification as primary failure causes, with LLM-based tools exhibiting additional unique failures. Finally, LLM-based methods incur substantial computational costs–hundreds of thousands to hundreds of millions of tokens (cost up to more than $1,000) and multi-hour to multi-day runtimes. Overall, our findings underscore critical limitations in the robustness, reliability, and scalability of current LLM-based detectors. We ultimately summarize a set of implications for future research toward more effective and practical project-scale vulnerability detection.

***Index Terms*—Vulnerability detection, Large language model, Empirical study.**

I. Introduction

With the growth of software complexity and deployment scale, the impact of security vulnerabilities becomes more severe. As a result, automated vulnerability detection has become a critical component of modern software development and maintenance. Broadly speaking, existing vulnerability detection approaches can be categorized into static and dynamic techniques [1], [2]. Among them, static vulnerability detection has been widely adopted by developers because it can be seamlessly integrated into continuous integration pipelines, operates as long as the code is parsable or compilable, and has good scalability [3].

Traditional static vulnerability detection approaches [4]–[10] typically encode expert knowledge as manually engineered rules, patterns, and data-flow templates that search for known classes of security problems, such as unsafe data flows from untrusted sources to sensitive sinks [11] or misuse of security-critical APIs. Although these analyzers are widely used in practice, they require substantial manual effort to design and maintain their rule sets, which is time-consuming and labor-intensive [3], [12], [13]. Moreover, their reliance on predefined patterns inherently limits the generalization and coverage for previously unseen or evolving vulnerability types [14], [15].

Recent advances in Large Language Models (LLMs) have sparked growing interest in LLM-based vulnerability detection. Trained on large corpora of code and natural language, LLMs can interpret code semantics, summarize complex logic, and reason about potential security risks beyond simple syntactic patterns. This has led to an emerging line of LLM-based vulnerability methods [16]–[20]. Unlike traditional tools, these methods feed source code directly to LLMs and rely on the models' reasoning capabilities instead of manually engineered program analyses. However, most of these tools focus on function-level or even smaller hunk-level code snippets, where LLMs are given relatively simple prompts to decide whether a vulnerability is present. This limited context and simplistic prompting make the detection particularly sensitive to LLM-induced hallucinations and inconsistent reasoning [21].

More recently, a growing line of work has proposed LLM-based vulnerability detectors that combine the semantic reasoning capabilities of LLMs with traditional static analysis workflows to scale vulnerability detection to the project level. These approaches either orchestrate existing static analysis tools with LLMs or use LLMs to emulate static analysis reasoning processes such as dependency and data-flow tracing, including specially designed agent pipelines such as RepoAudit [22], LLMDFA [23], LLM-assisted checker generators such as KNighter [24], IRIS [13], LLMxCPG [25], and control-flow guided methods such as INFERROI [26]. Although these tools demonstrate promising results, their evaluations are conducted on disparate datasets using inconsistent and limited settings and metrics, hindering a fair and systematic comparison of their effectiveness in real-world scenarios.

Several studies [27]–[35] have examined the static vulnerability detection capabilities of LLMs, but they either focus

Fengjie Li, Jiajun Jiang are with School of Computer Software, College of Intelligence and Computing, Tianjin University, Tianjin, China; Dongchi Chen is with International Joint Institute of Tianjin University, Fuzhou, China; Yingfei Xiong is with the Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education, School of Computer Science, Peking University, Beijing, China. (E-mail: fengjie@tju.edu.cn; jiangjiajun@tju.edu.cn; chendongchi@tju.edu.cn; xiongyf@pku.edu.cn).

primarily on simple prompt-based adaptation of LLMs [27]–[29] or report detection metrics only on function- or hunk-level datasets [31]–[35], thereby providing limited evidence of their effectiveness when working at the project scale. The exploration of these more specialized LLM-based detectors across different vulnerability types on real-world projects remains limited, leaving the following several key aspects insufficiently explored: **1) Effectiveness and complementarity on real-world projects:** These methods were originally evaluated on relatively limited benchmarks and adopt different evaluation settings. As a result, it remains unclear how robust and generalizable these methods perform when applied to large-scale real-world projects that contain diverse vulnerability types and complex project-specific APIs, and how much cross-tool complementarity they can actually provide in practice. **2) Comprehensive analysis of failure root causes:** Prior studies on LLM-based vulnerability detection primarily report aggregate metrics (e.g., precision, recall, F1) without systematically analyzing the factors that affect their practical usefulness. In particular, the root causes of false positives are rarely categorized or quantified, leaving practitioners with limited guidance on how to improve these tools. **3) Scalability & Overhead:** One primary concern for LLM-based methods is their cost and efficiency. However, existing work rarely measures the full computational overhead of LLM-based vulnerability detectors on real-world projects, such as LLM token usage and time overhead across different projects [27]. Consequently, there is a pressing need for a systematic study on real-world scenarios that jointly examines the effectiveness, failure root causes, and cost characteristics of these specialized LLM-based vulnerability detectors.

To address these gaps, this paper presents a comprehensive empirical study of specialized project-scale LLM-based vulnerability detectors across multiple CWE types, programming languages, and workflows, aiming to systematically analyze their strengths and weaknesses and to provide insights for more robust and efficient vulnerability detection techniques. Specifically, we evaluate five recent and representative LLM-based vulnerability detectors that are publicly available, span diverse design workflows, together with two widely used general agents, and four traditional static analyzers. Our evaluation spans two complementary settings: (1) a controlled benchmark of **265 already-known real-world vulnerabilities curated from existing benchmarks, covering 8 CWE types in C/C++ and Java**, which we use to assess the vulnerability detection capability of these methods systematically, and (2) **24** actively maintained open-source projects/modules, which we use to evaluate their practical usability in real-world scenarios. On the latter, we first analyzed 6,442 sampled warnings using Codex-assisted labeling with stratified manual validation and classified the root causes of 5,896 false positives using a taxonomy derived from 355 manually inspected reports (took more than 150 human hours), and the root-cause labels were further evaluated through stratified manual validation. For each tool, we systematically study its detection effectiveness, analyze the underlying causes of false positives, and measure the overhead.

Our results reveal several key findings. **First, general agent methods achieve the highest recall, but still struggle with complex code.** And both LLM-based and traditional tools suffer from incomplete API modeling, and providing precise project-specific API specifications substantially improves detection performance. Nevertheless, across this controlled dataset, while different methods present complementary results, the LLM-based methods still uncover more unique vulnerabilities than the traditional methods. **Second, both LLM-based and traditional methods exhibit a high false discovery rate.** Both LLM-based and traditional methods produce a large number of false alarms, with the lowest overall SFDR being 9.09% for C/C++ and 31.82% for Java. We derive the taxonomy from 355 manually inspected reports, use Codex to classify the root causes of 5,896 false positives, and perform stratified manual validation of the generated labels. The results show that truncated interprocedural context, incorrect program-point classification, and overlooked key program points in complex context are the dominant root causes, accounting for 861, 3734 and 543 false positives in our study, respectively. **Finally, project-scale LLM-based detection remains computationally expensive.** Our cost analysis shows that project-scale LLM-based detection can require between hundreds of thousands and hundreds of millions of tokens per project/module (e.g., up to **225,493.97K** input tokens and **38,078.26K** output tokens with Claude Sonnet 3.5), incur substantial monetary costs (up to $1247.66 per project with the same model), and take multi-hour to multi-day runtimes (e.g., up to **4,638.00** minutes with GPT-4), making scalability a critical bottleneck.

In summary, this work makes the following contributions:

- We conduct, to our knowledge, the first empirical study that compares **5** specialized LLM-based vulnerability detection tools with **2** general agents and **4** traditional static analyzers at the project scale.
- We systematically evaluate the detection effectiveness and cross-tool complementarity of these tools on **265** known vulnerabilities from **8** CWE types and **24** open-source projects/modules.
- We construct a taxonomy of false positive root causes for both LLM-based and traditional methods. We first manually label **355** sampled reports, then use Codex to label the remaining reports, and finally perform sampling-based human validation of the generated labels. By this process, we quantify how different design choices (e.g., data-flow construction, source/sink inference, and prompt following) contribute to these failures.
- We measure the computational overhead of LLM-based methods in terms of token usage, financial cost and runtime, and discuss the implications for practical deployment and future research on more effective and scalable analysis workflows.
- To foster reproducibility and future research, we release all experimental artifacts, including evaluation scripts, prompts, taxonomy labels, and detailed statistics, on our project homepage: https://github.com/Feng-Jay/LLM4Security.

II. Background and Related Works

A. Large Language Models

Recent advances in Large Language Models (LLMs) have significantly expanded their capability in Software Engineering (SE) fields. Modern foundation models (e.g., ChatGPT [36], Claude [37], and Gemini [38]) are trained on large corpora of source code and natural language, enabling them to generalize across diverse downstream tasks such as code summarization [39], program synthesis [40], and vulnerability repair [41]. With in-context learning, LLMs can be adapted to downstream tasks through few-shot examples [42] or chain-of-thought (COT) prompting [43], without requiring fine-tuning. In addition, emerging multi-agent frameworks further enhance LLM utility by decomposing complex tasks into coordinated agents, each responsible for localized subtasks, providing a scalable mechanism to integrate external tools and structured workflows [44]. These advances have recently motivated a growing number of LLM-based vulnerability detectors that aim to leverage the semantic reasoning capabilities of LLMs for static vulnerability detection.

B. Static Vulnerability Detection

Static vulnerability detection is a long-standing research topic in software security, which aims to detect vulnerabilities by analyzing source code or intermediate representations without executing the program [1]. Modern static analyzers detect vulnerabilities through pattern matching, dataflow analysis, and symbolic execution. Guided by predefined rules/checkers/patterns, they analyze vulnerability-relevant program information, such as data flows from untrusted inputs (sources) to security-sensitive operations (sinks), pointer states, resource lifecycles, and shared-memory accesses and synchronization. Representative tools include CodeQL [5], which treats code as a relational database and detects vulnerabilities using a declarative domain-specific language (DSL). Semgrep [6] performs fast pattern-based detection through syntactic and structural matching. SpotBugs [4], which performs bytecode-level pattern and dataflow checks for Java. Other widely used tools include joern [9], SonarQube [8], CodeSonar [45], CodeGuru [7], Infer [46]. Despite their good scalability, these tools rely heavily on manually designed rules, predefined API models, and language-specific vulnerability templates, which limit their ability to detect previously unseen or evolving vulnerability patterns.

Recently, numerous learning-based and LLM-based vulnerability detectors [16]–[20], [47]–[53] have been proposed to overcome these limitations. These methods include context-enhanced prompting, in-context learning, and RAG-based techniques that retrieve relevant code or vulnerability knowledge to improve LLM-based reasoning. Representative examples include GRACE [16], prompt-enhanced vulnerability detection [18], and RAG-based methods such as Vul-RAG and DLAP [52], [53]. Although these approaches demonstrate promising performance on benchmark datasets, they are often evaluated at function- or hunk-level granularity. In particular, these methods perform detection on individual functions rather than scanning complete repositories and producing project-level warning reports. Therefore, their results may not directly reflect tool effectiveness, false-positive behavior, or scalability in real-world project-scale settings [54], [55]. More recently, LLM-based vulnerability detection frameworks have been integrated into static analysis workflows to scale vulnerability detection to the project level. Examples include RepoAudit [22] and LLMDFA [23], which use multi-agent workflows or iterative prompting to analyze entire repositories. KNighter [24], IRIS [13] and LLMxCPG [25], which utilize LLMs to automatically synthesize static checkers. And INFERROI [26], which integrates control-flow slicing with LLM-guided root cause inference. These specialized LLM-based tools indicate a transition toward project-scale vulnerability analysis. However, systematic evaluation of their effectiveness, failure modes, and scalability remains limited.

C. Existing Studies on LLM-based Vulnerability Detection

Several studies have already explored the use of LLMs for vulnerability detection. Wen et al. [27], Purba et al. [28], and Lin et al. [29] evaluate various LLMs at both function- and project-level using real-world CVEs. However, rather than analyzing full software systems, their evaluation is restricted to dependency-related functions extracted around the specific modification locations of each CVE. The resulting contexts are often incomplete: they cover only a subset of files deemed related to the vulnerability and focus narrowly on code fragments surrounding the known issue. Consequently, such settings cannot faithfully reflect the false-positive behavior and practical effectiveness of LLM-based methods when applied to real-world projects. Similarly, Yildiz et al. [30] benchmarked multiple LLM-based workflows for vulnerability detection at the commit level. Besides, although some studies [31]–[35] also explored LLM-based vulnerability detection from various perspectives, they primarily operate at function- or hunk-level and lack systematic evaluation in real-world project settings, leaving it unclear how well such methods generalize to large software systems with complex call graphs and project-specific APIs.

III. Study Design

A. Research Questions

To comprehensively evaluate the effectiveness of existing LLM-based vulnerability detectors and to further understand the key factors that affect their effectiveness, we primarily design and investigate the following four research questions in this study:

RQ1: How effective are LLM-based methods in detecting known vulnerabilities in the controlled benchmark? In this RQ, we curate a controlled vulnerability dataset by collecting and filtering existing peer-reviewed benchmarks [13], [56], [57] and then evaluate the selected methods on it. Our analysis aims to provide a clear and direct assessment of the detection capabilities of these methods, identifying the key factors that influence their effectiveness.

RQ2: How reliable are LLM-based vulnerability detectors when applied to real-world projects/modules in terms of false discovery rates? To further investigate the practical

TABLE I: Studied vulnerability detection methods in this paper

Category	Method	LLM Backbone	Languages	Evaluated CWE Type	Workflow	API Model Source
Specialized LLM-based	RepoAudit [22]	Claude Sonnet 3.5 / GPT-5.4	C/C++	CWE-401, CWE-416, CWE-476	Multi-Agent	Manual
	Knighter [24]	O3-mini / GPT-5.4	C/C++	CWE-401, CWE-416, CWE-476	LLM+CSA [10]	LLM-inferred
	IRIS [13]	GPT-4 / GPT-5.4	Java	CWE-022, CWE-078, CWE-079, CWE-094	LLM+CodeQL	LLM-inferred
	LLMDFA [23]	GPT-4 / GPT-5.4	Java	CWE-078, CWE-079	Multi-Agent	LLM-inferred
	INFERROI [26]	GPT-4 / GPT-5.4	Java	CWE-772	LLM+CFG	LLM-inferred
General Agents	Codex [58]	GPT-5.4	C/C++, Java	All above	Multi-Agent	LLM-inferred
	Claude Code [59]	Claude Sonnet 4.6	C/C++, Java	All above	Multi-Agent	LLM-inferred
Traditional	SpotBugs [4]	-	Java	CWE-022, CWE-078, CWE-079, CWE-094, CWE-772	Bytecode analysis	Built-in
	CSA [10]	-	C/C++	CWE-401, CWE-416, CWE-476	Symbolic execution	Built-in
	CodeQL [5]	-	C/C++, Java	All above	Predicate-based querying	Built-in
	Semgrep [6]	-	C/C++, Java	CWE-401, CWE-416, CWE-476 CWE-022, CWE-078, CWE-079, CWE-094	Pattern matching	Built-in

effectiveness of LLM-based vulnerability detectors, we apply them to a set of recent real-world projects. In this RQ, rather than comparing the vulnerability detection capability (recall), we focus on the number of warnings these methods report and the number of false positives. This provides an empirical understanding of how these tools perform in realistic development environments.

RQ3: What are the root causes of false positives? In this RQ, we combine manual taxonomy construction, LLM-assisted classification, and manual validation to analyze the causes behind incorrect vulnerability reports produced by these tools in real-world projects/modules. This analysis aims to better understand the limitations of current LLM-based detectors in real-world scenarios and to offer implications for future research and practical tool development.

RQ4: What is the overhead of applying LLM-based vulnerability detectors at the project scale? Compared with traditional tools, a key concern for LLM-based methods lies in their cost and efficiency. In this RQ, we investigate the runtime and token usage of LLM-based vulnerability detectors to evaluate their scalability and practicality for real-world project-level analysis.

B. Evaluated Methods

As shown in Table I, we selected five of the latest and most representative specialized LLM-based methods, two general agents and four traditional static analysis methods that (i) are publicly available, (ii) cover a diverse range of workflows, (iii) support project-scale analysis and can produce concrete vulnerability warnings and (iv) can be readily applied across different projects and programming languages. This selection enables us to assess the effectiveness of LLM-based approaches as well as their advantages, limitations, and potential complementarity with existing traditional tools in real-world scenarios. Based on the above criteria, we focus on methods that can directly scan complete repositories and produce project-level warning reports. Therefore, we do not include prompt-only, in-context learning, or RAG-based methods such as GRACE, Vul-RAG, and DLAP as direct baselines, because they are mainly evaluated at the function level and do not directly provide comparable repository-level warning reports in our setting. We also do not include LLMxCPG as a direct baseline because its workflow requires local model finetuning, inference, and non-trivial adaptation to our CWE-specific evaluation setting. Moreover, its CPG-based checker-generation design is close to the checker-synthesis family already represented by KNighter. For specialized LLM-based methods, considering that different backbone LLMs may affect detection capability, we evaluate each method using both the model adopted in its original paper and the strongest recent model available at the time of our study (GPT-5.4) to control for backbone-model effects. These methods are selected to represent a diverse range of detection workflows, LLM backbones, supported programming languages, and targeted CWE types[1]. In the following, we briefly introduce each of them.

1) Specialized LLM-based Methods:

- **RepoAudit [22]** A multi-agent framework initiates scanning from predefined source points and systematically explores program paths by identifying critical callee or caller functions with the assistance of LLMs. When a path reaches a predefined sink point, it reports a potential vulnerability. The reported paths are then subsequently validated by LLMs to reduce false positives and improve the overall performance.
- **KNighter [24]** An LLM-based checker generation framework that automatically generates Clang Static Analyzer (CSA) [10] checkers from a vulnerability fixing commit. The generation process includes few-shot prompting, iterative regeneration with syntax validation and repair, and refinement steps to ensure high detection precision. Since KNighter relies on the CSA, it can detect multiple types of vulnerabilities in C/C++.
- **IRIS [13]** IRIS utilizes LLMs to automatically identify and label all methods and their corresponding parameters within a project as potential sources or sinks. This information is then embedded into predefined CodeQL query templates to detect possible vulnerabilities. The paths flagged as potentially vulnerable by these queries are then re-evaluated by LLMs to reduce false positives.
- **LLMDFA [23]** An agent-centric method initiates scanning from source program points identified by the LLM. Unlike RepoAudit, which will explore program paths on demand, LLMDFA will explore all possible paths. When a path reaches a sink point identified by the LLM, it reports a potential vulnerability path. These paths are further validated for reachability by using LLMs or the Z3 solver [60].

[1]CWE categories and definitions referenced in this paper are based on the official MITRE classification at https://cwe.mitre.org/.

- **INFERROI [26]** This method is specifically designed to detect resource leak vulnerability (CWE-772). It first employs LLMs to infer the intent of program statements (resource acquisition, release, or check). Then it explores program paths based on the control flow graph (CFG). If a path exists where a resource is acquired but never released, it is reported as a potential vulnerability.

2) General Agents:

- **Codex** [58] and **Claude Code** [59] are general-purpose coding agents capable of repository inspection, command execution, file editing, and multi-step reasoning from natural-language instructions. We use them as general-agent baselines to evaluate whether broadly capable coding agents can perform vulnerability detection without a specialized security-analysis pipeline.

3) Traditional Static Methods:

- **CodeQL [5]** A semantic, predicate-based querying static analysis framework widely used in industry for vulnerability detection. It models source code as a relational dataset and uses logic-style queries to identify relevant patterns across control-flow and data-flow paths. It supports multiple programming languages, including C/C++ and Java, and provides a rich collection of built-in security queries covering a broad range of CWEs. In this study, we apply CodeQL to C/C++ and Java vulnerability types supported by the evaluated LLM-based vulnerability detectors, enabling a consistent and comparable assessment across all methods.
- **Semgrep [6]** A lightweight, pattern-based static analysis tool that identifies vulnerabilities using rule-based matching over Abstract Syntax Trees (ASTs). It also supports many programming languages, including C/C++ and Java, and offers an extensive rule ecosystem maintained by the community and security researchers. Similarly, we apply Semgrep to C/C++ and Java vulnerabilities that are supported by both the LLM-based tools and the available Semgrep rule set.
- **SpotBugs [4]** A bytecode-level static analysis tool for Java that detects bug patterns and security-relevant defects through a collection of predefined detectors. SpotBugs analyzes compiled Java bytecode and reports potential issues such as null dereferences, resource leaks, and unsafe API usages. In this study, we apply SpotBugs together with Find Security Bugs [61], its recommended security plugin, to Java vulnerability types covered by their detector sets, providing a traditional Java-specific baseline.
- **Clang Static Analyzer (CSA) [10]** is a path-sensitive symbolic-execution-based static analyzer for C/C++ that tracks program states along feasible execution paths and reports defects such as null dereferences, use-after-free errors, and memory leaks. We include CSA as a C/C++ static-analysis baseline and apply it to vulnerability types supported by its analysis rules.

C. Datasets

As illustrated in our research questions, we evaluate the selected methods in two scenarios: a controlled dataset containing projects that are verified to contain known vulnerabilities, and a real-world dataset composed of the latest open-source projects/modules. We select eight CWE categories based on implementation support (Table I) and benchmark availability. They cover path traversal and injection, memory safety, and resource management. We evaluate methods within their supported categories: for example, INFERROI targets resource leaks, while IRIS uses CWE-specific query templates. Extending these tools may require substantial effort and new analysis logic, which we leave to future work. Dataset details appear in Table II and Table III.

- **Controlled dataset:** We construct a controlled dataset by consolidating vulnerabilities from three peer-reviewed benchmarks, providing a controlled setting to assess the detection effectiveness of the evaluated methods: ReposVul [56], CWE-Bench-Java [13], and JLeaks [57].
  For C/C++, we use ReposVul, curated using LLMs and static analysis and containing 6,134 CVEs across 236 CWE types and 1,491 projects. We select vulnerabilities from the three repositories with the most cases for each evaluated CWE, limiting build and configuration effort across repository versions.
  For Java, CWE-Bench-Java provides manually curated Java vulnerabilities from real-world projects. From this benchmark, we select vulnerable cases from Maven projects to guarantee successful building and analysis. JLeaks is a high-quality benchmark focusing on Java resource leak vulnerabilities and has been manually validated. From JLeaks, we randomly sample 50 representative cases. All selected cases from these three sources are further manually checked to ensure correctness, validity, and successful compilation.
- **Real-world projects/modules:** To complement the controlled dataset, we construct a real-world dataset by selecting actively maintained open-source projects/modules that have historically exhibited vulnerabilities of the studied CWE types. As summarized in Table III, these projects/modules span both C/C++ and Java and cover diverse domains such as operating systems, multimedia processing, networking, data platforms, and web frameworks. We use the latest versions of actively maintained software with historical vulnerabilities in the studied CWE types, including Linux, ImageMagick, and Vim for C/C++ [56], and XStream, Jenkins, and Keycloak for Java [13]. Of the 24 project/module entries, 21 use commits postdating the reported August 2025 knowledge cutoffs of GPT-5.4 [62] and Claude Sonnet 4.6 [63]. This dataset provides a realistic and diverse foundation for evaluating the scalability and practical effectiveness of the evaluated methods.

D. Evaluation Metrics

To evaluate the effectiveness of the studied methods, we adopt several standard detection metrics by following existing studies [27]–[35], and additionally consider tool costs in terms of token usage and time overhead.

Controlled dataset. For the curated controlled dataset, each vulnerable project is labeled with the actual vulnerable program point(s). Consequently, we consider a vulnerability successfully detected by a tool as long as it reports the associated vulnerable point(s) (*a.k.a*, true positive: TP), otherwise, the

TABLE II: Dataset overview and illustrative CWE examples

Language	CWE	Cases	Illustrative Example	Benchmark
C/C++	CWE-401	50	Missing `free` after `malloc`	ReposVul [56]
	CWE-416	25	Reading a freed buffer via `memcpy`	
	CWE-476	35	Dereferencing `p` when `p == NULL`	
Java	CWE-022	49	Path traversal through `FileInputStream`	CWE-Bench-Java [13]
	CWE-078	11	Injected commands executed by `Runtime.exec`	
	CWE-079	28	Unescaped HTML output via `JspWriter.print`	
	CWE-094	17	Injected code executed by `ScriptEngine.eval`	
	CWE-772	50	Missing `close` after `FileReader` creation	JLeaks [57]

TABLE III: Real-world projects/modules used in this study

CWE	Repository Name	Commit	Size (LOC)	Stars
CWE-401	linux/sound	6093a68	1,253,234	207K
	linux/mm	6093a68	133,952	207K
	ImageMagick	3bf1076	680,878	14.9K
CWE-416	linux/net	6093a68	962,143	207K
	linux/drivers/net	6093a68	3,924,621	207K
	vim	8feaa94	1,192,925	39.3K
CWE-476	linux/drivers/peci	6093a68	1,715	207K
	gpac	c6a72c3	859,528	3.1K
	bitlbee	8af06ca	33,650	632
CWE-022	OpenOLAT	df53b85	1,907,083	394
	spark	1973e40	11,941	9.7K
	Dspace	3b24801	448,030	1K
CWE-078	xstream	a22d3af	76,339	752
	workflow-cps-plugin	e4b9517	238,588	179
	tika	2b38ed1	228,309	3.4K
CWE-079	xwiki-platform	fc73498	5,553,098	1.2K
	jenkins	69d5a54	300,491	24.7K
	keycloak	4f55b9b6	6,033,855	31K
CWE-094	onedev	b8a4d7cd	552,486	14.5K
	activemq	56ea235e	611,964	2.4K
	cron-utils	bac6e866	16,244	1.2K
CWE-772	sql2o	744b85b3	8,027	1.2K
	RxJava	f07765ed	325,394	48.4K
	jsoup	acafbcf3	38,128	11.3K

vulnerability is missed by the tool (*a.k.a*, false negative: FN). To measure precision, we classify reported warnings as false positives (FPs) if their associated data flows involve known vulnerable files but do not match the data-flow patterns of known vulnerabilities, as defined by the IRIS setup [13]. We made this decision for two reasons. First, as our experiments will show, the studied methods have relatively high false positive rates. Consequently, we are confident that many of these FPs are actually not vulnerabilities while in theory they have the potential to be other unknown genuine vulnerabilities. Second, in a later experiment, we will analyze a large set of warnings reported by the same tools on other real-world projects or modules. Therefore, to keep the experiments manageable and avoid repetitive work, we focus on the known vulnerabilities in this controlled dataset. We then compute recall and precision as follows:

$$\text{Recall} = \frac{\text{TP}}{\text{TP}+\text{FN}} \times 100\%. \quad \text{Precision} = \frac{\text{TP}}{\text{TP}+\text{FP}} \times 100\%.$$

Real-world projects/modules. For these real-world open-source projects/modules, it is unknown whether there are vulnerabilities in them. For each tool and project/module, we therefore report: **1) #Reports**, the total number of warnings produced, **2) #Files**, the number of distinct source files that contain at least one warning, and **3) #Sampled FPs**, the number of FPs (benign instances incorrectly reported as vulnerabilities by the tools) among the sampled warnings. Since manually validating a warning is time-consuming and labor-intensive, it often requires inspecting project-specific APIs, transitive dependencies, and relevant control and data flows across the entire project/module. Therefore, we utilize Codex to automatically label whether each reported warning is a true vulnerability or not. Based on these labels, we estimate the Sampled False Discovery Rate (SFDR) as

$$\text{SFDR} = \frac{\#\text{Sampled FPs}}{\#\text{Sampled Reports}}$$

To facilitate a comprehensive root cause analysis of false positives reported by existing approaches, up to 10 warnings per tool per project were initially sampled by two authors. The identified causes were then summarized to formulate an initial taxonomy and corresponding classification codebook. This manual effort involved the examination of **355** sampled warnings, consuming more than **150** person-hours. Based on this taxonomy, Codex was subsequently used to label the remaining warnings. To verify the reliability of these automated results, Codex's outputs, for both the labeling and the root-cause classification phases, were manually checked against a sampled subset of instances. The details will be introduced in Section V-B and V-C.

Token usage and time overhead. For RQ4, we quantify the computational overhead of LLM-based methods by measuring the input and output token consumption of each tool, financial cost, as well as the end-to-end detection time per project/module (Section V-D).

IV. Experimental Setup

For specialized LLM-based vulnerability detection methods, we directly use their released replication packages and follow hyperparameters (e.g., function call-chain exploration depth, temperature, and top-p) recommended in the original papers, imposing no additional timeouts, token or monetary budgets. To assess the impact of model choice, we evaluate each method with both its original backbone model and the latest model,

GPT-5.4. All experiments are conducted via the official OpenAI and Anthropic Claude APIs [62]–[66]. For RepoAudit that predefined vulnerability-related program points, we use their default configurations on both datasets. For IRIS, LLMDFA, and INFERROI, we use the API lists generated by their respective LLMs. For KNighter, we generate CSA checkers for each CWE using the corresponding bug-fixing patches from its original study (18 patches in total).

For the general-agent baselines, Codex and Claude Code, we use their official CLI interfaces with built-in tools and skills enabled, and disable all Internet search capabilities to ensure a controlled evaluation setting. All experiments are conducted through the official OpenAI and Anthropic APIs. The evaluation prompt is available at our project[2].

For traditional static vulnerability detection methods, we manually curate the predefined rule set by selecting only those rules that are relevant to the target CWE types evaluated in this study. Specifically, for CodeQL, we finally selected 48 predefined rules that are relevant to our target CWE types from the official CodeQL rule set, such as `CWE-401/MemoryLeakOnFailedCallToRealloc.ql` and `Critical/MemoryMayNotBeFreed.ql` for CWE-401. For Semgrep, we finally selected 59 predefined rules that are relevant to our target CWE types from the official rule sets (e.g., the official `XSS` rules for CWE-079). For SpotBugs, together with the Find Security Bugs [61] plugin, we iterated over the detectors and bug patterns from its official list and the Find Security Bugs plugin, mapped them to our target Java CWE types, and finally obtained **21** relevant predefined patterns. For CSA, we selected **12** built-in C/C++ checkers relevant to our target CWE types, including memory allocation/deallocation-related checkers, nullness checkers, and `StdCLibraryFunctions` checkers. More details about the selected rules/patterns/checkers are available in our Appendix B and on our homepage [67]. Although different rules/patterns/checkers may produce reports with varying severity levels (e.g., some are marked as errors and others as recommendations), we treat all reports equally in this study.

All experiments are conducted on a local server equipped with dual Intel Xeon 6388 CPUs, 512 GB of RAM, running Ubuntu 20.04.6 LTS. All LLM API requests, as well as CodeQL, Semgrep, SpotBugs and CSA analyses, are executed on this server in our experiments.

V. Result Analysis

A. RQ1: Detection Effectiveness on the Controlled Dataset

Results on C/C++ vulnerabilities. Table IV demonstrates the detection results of evaluated methods on different CWE types. Overall, the two general agent methods achieve an average recall of 59.55% (131/220) and an average precision of 32.19% (131/407). In particular, Codex detects the largest number of vulnerabilities. In contrast, among the two specialized LLM-based methods, KNighter fails to detect any vulnerabilities in these three CWE categories, while RepoAudit$_{GPT-5.4}$ detects only two. Their performance is worse than that of some traditional tools, such as CodeQL, which detects 3 CWE-401 and 12 CWE-476 vulnerabilities. This consistently low recall across specialized LLM-based methods suggests limited generalizability beyond their original evaluation settings. In particular, for linux kernel–related vulnerabilities, RepoAudit was evaluated on only 4 known intra-function cases in its original study, while KNighter did not report recall against known ground truth. These limitations motivate us further to investigate the underlying causes of their missed detections and the substantial performance gap between specialized LLM-based methods and general-purpose agents on this dataset.

TABLE IV: Effectiveness of evaluated tools on the C/C++ controlled dataset.

CWE	Method	#Warn.	#Det.	Prec.	Rec.
CWE-401 Total: 50	RepoAudit$_{Sonnet3.5}$	0	0	–	0.00
	RepoAudit$_{GPT-5.4}$	0	0	–	0.00
	KNighter$_{O3-mini}$	0	0	–	0.00
	KNighter$_{GPT-5.4}$	0	0	–	0.00
	Codex$_{GPT-5.4}$	81	34	41.98	68.00
	Claude Code$_{Sonnet4.6}$	100	40	40.00	80.00
	CodeQL	21	3	14.29	6.00
	Semgrep	0	0	–	0.00
	CSA	1	0	0.00	0.00
CWE-416 Total: 25	RepoAudit$_{Sonnet3.5}$	33	0	0.00	0.00
	RepoAudit$_{GPT-5.4}$	23	0	0.00	0.00
	KNighter$_{O3-mini}$	0	0	–	0.00
	KNighter$_{GPT-5.4}$	0	0	–	0.00
	Codex$_{GPT-5.4}$	22	10	45.45	40.00
	Claude Code$_{Sonnet4.6}$	51	9	17.65	36.00
	CodeQL	0	0	–	0.00
	Semgrep	2	0	0.00	0.00
	CSA	11	0	0.00	0.00
CWE-476 Total: 35	RepoAudit$_{Sonnet3.5}$	7	0	0.00	0.00
	RepoAudit$_{GPT-5.4}$	55	2	3.64	5.71
	KNighter$_{O3-mini}$	0	0	–	0.00
	KNighter$_{GPT-5.4}$	0	0	–	0.00
	Codex$_{GPT-5.4}$	60	22	36.67	62.86
	Claude Code$_{Sonnet4.6}$	93	16	17.20	45.71
	CodeQL	1110	12	1.08	34.29
	Semgrep	0	0	–	0.00
	CSA	11	0	0.00	0.00

Manual analysis (Fig. 1) identifies **incomplete API modeling and incomplete candidate exploration as major causes of missed detections**. Incomplete API modeling occurs when rules, generated checkers, or LLM reasoning omit relevant APIs or their roles, such as resource allocation and release. For source/sink-based analyses, this includes omitting a source, a sink, or both. Incomplete candidate exploration occurs when relevant APIs are recognized, but vulnerability-relevant call sites are not examined. Our analysis attributes 108/108, 97/110, 40/44, and 40/45 missed vulnerabilities for RepoAudit$_{GPT-5.4}$, KNighter$_{GPT-5.4}$, Codex, and Claude Code, respectively, to these two issues. Moreover, since KNighter generates CSA checkers from existing patches, its checkers encode not only data-flow information but also patch-

[2]https://github.com/Feng-Jay/LLM4Security/blob/master/src/core/codex.py

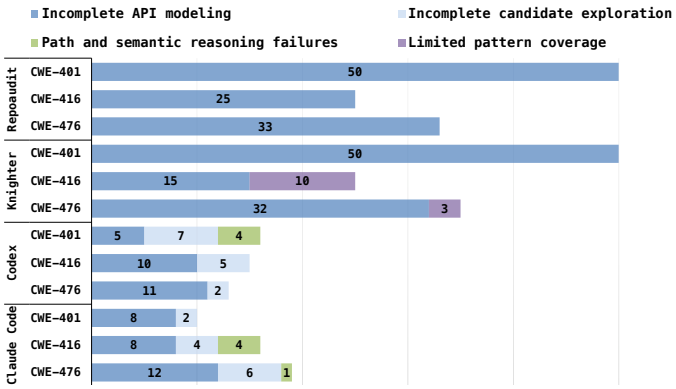


Fig. 1: Reasons for undetected C/C++ vulnerabilities.

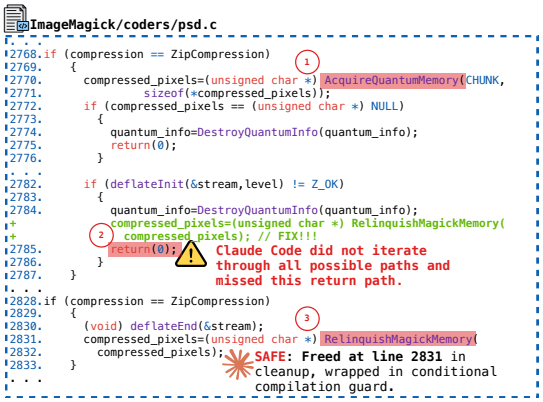


Fig. 2: Claude Code missed vulnerability due to incomplete path exploration.

specific patterns. KNighter additionally misses 13 vulnerabilities due to limited pattern coverage. Its generated checkers include the relevant APIs, but their encoded pattern conditions do not cover the vulnerable usage in these cases. For Claude Code and Codex, several additional misses arise from incorrectly handling complex control-flow graphs.

Here is an example shown in Fig. 3, the target CWE-401 vulnerability is triggered because the function directly returns in the `if-true` branch (line 31) without releasing the `msg` object allocated by the project-specific API `nlmsg_new` (line 25). However, all four methods shown in the figure cannot detect it. For example, Claude Code and Codex first inspect the source file and execute commands such as `grep` and `rg` to search for potential APIs. However, our analysis of their sessions shows that neither identifies the project-specific allocation API `nlmsg_new`, likely because of the file's complexity and size (more than 17,000 lines of code). Consequently, their subsequent reasoning fails to detect the vulnerability. More generally, we collect the files containing the source and sink APIs of both recalled and not recalled vulnerabilities and measure their LOC. As shown in Fig. 4, missed vulnerabilities tend to involve larger files, suggesting that file size and complexity hinder general-purpose agents' ability to detect vulnerabilities through simple file inspection and API searches.

Similarly, CSA checkers generated by KNighter focus exclusively on matching limited deallocation functions (e.g., `kfree`), while this design enables customization, extending these checkers in practice requires familiarity with CSA libraries, macros, and the tool's analysis workflow, and typically involves writing and debugging substantial C code (often more than 100 lines). By default, RepoAudit models only a limited set of common allocation and deallocation APIs. Supporting project-specific APIs requires developers to understand the relevant AST structures and libraries and to configure and refine the corresponding modeling rules, thereby increasing the barrier to practical adoption.

TABLE V: CWE-401 detection performance with and without perfect allocation/deallocation API specifications.

Method	Default Det./Rec.	Perfect Det./Rec.	Δ Rec.
RepoAudit$_{@GPT-5.4}$	0 / 0.00%	24 / 48.00%	+48.00
Codex$_{@GPT-5.4}$	34 / 68.00%	45 / 90.00%	+22.00
Claude Code$_{*Sonnet4.6}$	40 / 80.00%	48 / 96.00%	+16.00
CodeQL	3 / 6.00%	28 / 56.00%	+50.00

Fig. 2 shows another example of path and semantic reasoning failures. The vulnerability is triggered because `compressed_pixels`, allocated at Line 2770, is not cleaned up before the early `return (0)` at Line 2785. However, Claude Code incorrectly treats the path as safe by relying on a later release at Line 2831, which is unreachable once the function returns early.

Results with API guidance. Given the substantial impact of API modeling, we conduct an additional API-guided experiment on CWE-401 vulnerabilities. Specifically, we provide RepoAudit$_{@GPT-5.4}$, Codex, Claude Code, and CodeQL with project-specific allocation and deallocation APIs while keeping all other settings unchanged. We exclude KNighter, CSA, and Semgrep because their checkers encode hard-coded, pattern-specific information and cannot be readily adapted to new API specifications. As shown in Table V, perfect API specifications substantially improve all four methods. CodeQL exhibits the largest absolute recall improvement, increasing recall by 50.00 percentage points, followed by RepoAudit$_{@GPT-5.4}$ with 48.00 percentage points. The recall of Codex and Claude Code increases to 90.00% and 96.00%, respectively. These results confirm that incomplete API modeling is a major cause of missed detections.

Nevertheless, RepoAudit$_{@GPT-5.4}$ and CodeQL still miss 26 and 22 vulnerabilities, respectively, indicating that accurate API specifications alone cannot address limitations in subsequent data-flow and control-flow analysis. For RepoAudit$_{@GPT-5.4}$, the remaining misses mainly arise because it formulates vulnerability detection as a path-feasibility problem and uses a **fixed few-shot prompt** to ask the LLM to generate formatted path descriptions. Our manual inspection shows that these extracted paths are often incomplete or fail to be parsed correctly by the downstream analysis logic, causing true vulnerabilities to be missed. For CodeQL, although the supplied APIs improve the completeness of API modeling, the built-in queries are primarily centered on **stack-local** ownership and treat field or global assignments as ownership escapes. As a result, it often fails to report cases where

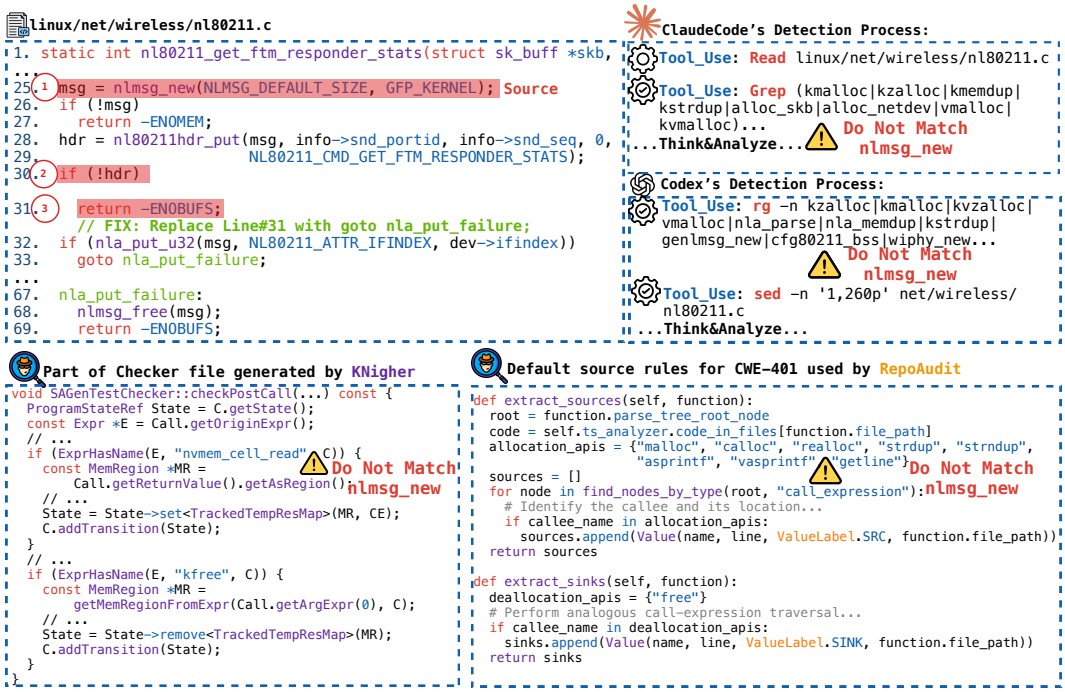


Fig. 3: Illustrating example of why these tools fail to detect the vulnerability.

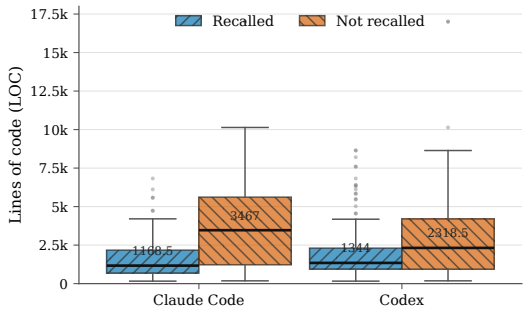


Fig. 4: LOC distribution for recalled vs. not-recalled C/C++ vulnerabilities

a field-owned object is not properly cleaned up along a specific error path. Thus, these results show that accurate API modeling is critical, but it is not sufficient on its own. Existing methods with relatively fixed detection logic can still constrain the capability of the underlying model. For example, RepoAudit$_{GPT-5.4}$ and Codex use the same model, yet Codex achieves higher recall in our setting. This suggests that workflow design, such as flexible agentic exploration versus fixed prompt-and-parsing pipelines, can substantially affect how effectively the model's reasoning capability is used.

Results on Java vulnerabilities. For Java vulnerabilities, effectiveness varies across tools and CWE types. Overall, the general-agent methods, Codex and Claude Code, detect the most vulnerabilities, achieving average recalls of 63.87% (99/155) and 81.94% (127/155), respectively. Traditional analyzers (CodeQL, Semgrep, and SpotBugs) perform worse, achieving average recalls of 14.19%, 6.67%, and 14.84%, respectively. For specialized LLM-based methods, IRIS$_{GPT-5.4}$ achieves strong performance. In contrast, LLMDFA$_{GPT-5.4}$ yields very few detections, identifying only 1 vulnerability. Manual inspection shows that, similar to the C/C++ results, most of LLMDFA's missed cases (38/39) arise because the source and sink APIs inferred by the LLMs do not match the actual project-specific APIs. The fundamental reason is that **LLMs struggle to generate accurate project-specific API models from limited prompt context.** In LLMDFA, few-shot prompting is used to guide the LLM to predict potential source-sink APIs, so the inferred API models tend to resemble the examples rather than fully tailored to the project. In contrast, IRIS systematically enumerates and analyzes all imported and locally defined APIs within the project, leading to more accurate and complete API models. Besides, for resource-leak vulnerabilities (CWE-772), INFERROI$_{GPT-5.4}$ achieves a recall of 66.00%, which is in line with its original paper and outperforms the traditional analyzers CodeQL and SpotBugs, which achieve recalls of 10.00% and 6.00%, respectively. It also performs comparably to Codex in recall.

Moreover, we observe that replacing the models originally used by these specialized LLM-based methods with a stronger recent model (GPT-5.4) does not substantially improve recall, similar to the results on C/C++ vulnerabilities. This suggests that their recall is largely constrained by the detection workflows designed by the methods themselves, rather than by model capability alone. However, stronger models do lead to notable precision improvements. For example, with GPT-5.4, IRIS improves its precision on CWE-078 from 6.06% to 20.00%, while INFERROI improves from 15.74% to 20.50%.

Complementary Analysis. To further investigate how different tools complement or overlap with each other, we analyze the overlap of vulnerabilities detected by traditional tools and

TABLE VI: Effectiveness of evaluated tools on the Java controlled dataset.

CWE	Method	Warn.	Det.	Prec.	Rec.	CWE	Method	Warn.	Det.	Prec.	Rec.
CWE-078 Total: 11	IRIS$_{GPT-4}$	33	2	6.06	18.18	CWE-022 Total: 49	IRIS$_{GPT-4}$	381	24	6.30	48.98
	IRIS$_{GPT-5.4}$	5	1	20.00	9.09		IRIS$_{GPT-5.4}$	305	23	7.54	46.94
	LLMDFA$_{GPT-4}$	21	0	0.00	0.00		Codex$_{GPT-5.4}$	63	37	58.73	75.51
	LLMDFA$_{GPT-5.4}$	0	0	–	0.00		Claude Code$_{Sonnet4.6}$	118	44	37.29	89.80
	Codex$_{GPT-5.4}$	5	2	40.00	18.18		CodeQL	199	16	8.04	32.65
	Claude Code$_{Sonnet4.6}$	12	5	41.67	45.45		Semgrep	2075	6	0.29	12.24
	CodeQL	1	0	0.00	0.00		SpotBugs	910	16	1.76	32.65
	Semgrep	5	1	20.00	9.09	CWE-094 Total: 17	IRIS$_{GPT-4}$	238	6	2.52	35.29
	SpotBugs	5	2	40.00	18.18		IRIS$_{GPT-5.4}$	51	4	7.84	23.53
CWE-079 Total: 28	IRIS$_{GPT-4}$	149	11	7.38	39.29		Codex$_{GPT-5.4}$	11	7	63.64	41.18
	IRIS$_{GPT-5.4}$	82	5	6.10	17.86		Claude Code$_{Sonnet4.6}$	38	15	39.47	88.24
	LLMDFA$_{GPT-4}$	9	0	0.00	0.00		CodeQL	0	0	–	0.00
	LLMDFA$_{GPT-5.4}$	5	1	20.00	3.57		Semgrep	50	0	0.00	0.00
	Codex$_{GPT-5.4}$	38	14	36.84	50.00		SpotBugs	15	2	13.33	11.76
	Claude Code$_{Sonnet4.6}$	107	18	16.82	64.29	CWE-772 Total: 50	INFERROI$_{GPT-4}$	197	31	15.74	62.00
	CodeQL	11	1	9.09	3.57		INFERROI$_{GPT-5.4}$	161	33	20.50	66.00
	Semgrep	42	0	0.00	0.00		Codex$_{GPT-5.4}$	75	39	52.00	78.00
	SpotBugs	16	0	0.00	0.00		Claude Code$_{Sonnet4.6}$	107	45	42.06	90.00
							CodeQL	10	5	50.00	10.00
							SpotBugs	4	3	75.00	6.00

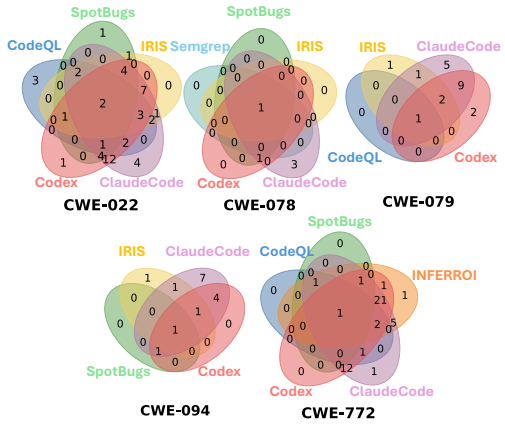


Fig. 5: Unique vulnerabilities detected by each tool

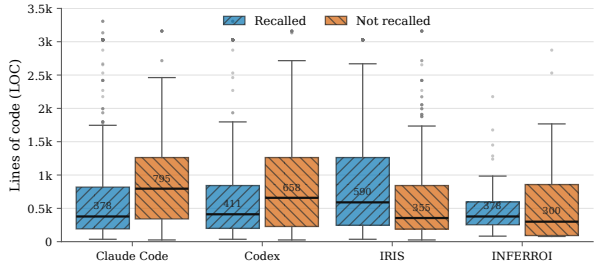


Fig. 6: LOC distribution for recalled vs. not-recalled Java vulnerabilities

LLM-based methods equipped with GPT-5.4 and Claude Sonnet 4.6. As shown in Fig. 5, LLM-based methods contribute a substantially larger portion of unique detections, highlighting their ability to surface project-specific or unconventional patterns that traditional tools do not cover. In particular, the two general-agent methods, Claude Code and Codex, account for most detections and exhibit substantial overlap, indicating that they often identify similar vulnerable patterns. Meanwhile, specialized LLM-based methods such as IRIS and INFERROI provide complementary coverage for specific CWE categories; for example, INFERROI detects many CWE-772 cases and overlaps substantially with both general agents. In contrast, traditional tools such as CodeQL, Semgrep, and SpotBugs yield very limited unique detections. For example, CodeQL contributes at most three unique detections on CWE-022, while most findings from traditional tools fall within the detection sets of LLM-based methods. These results suggest that, in our evaluated setting, these LLM-based methods show partial overlap with traditional static tools and may identify some vulnerabilities not detected by the traditional baselines.

Although Codex and Claude Code achieve strong recall, they still exhibit limitations when analyzing complex files. As shown in Fig. 6, we collect the LOC of files containing the source and sink APIs for both detected and missed vulnerabilities across Codex, Claude Code, IRIS, and INFERROI. For general-agent methods, files associated with detected vulnerabilities tend to have lower LOC than those associated with missed vulnerabilities, suggesting that file size and complexity remain key obstacles for agents that rely on direct source-code inspection and search. In contrast, specialized LLM-based methods are largely unaffected by this issue. This is because IRIS and INFERROI leverage traditional analysis tools or algorithms to reason over more compact representations, such as CFGs, rather than directly analyzing full source files, thereby mitigating the impact of large file size.

TABLE VII: Detection results of the evaluated tools on real-world C/C++ projects. In this table, #Warn. denotes the number of warnings, superscripts indicate SFDR sample sizes. – indicates that the SFDR cannot be calculated.

CWE	Project	RepoAudit$_{Sonnet3.5}$		Knighter$_{O3-mini}$		Codex$_{GPT-5.4}$		Claude Code$_{Sonnet4.6}$		CodeQL		Semgrep		CSA	
		#Warn.	SFDR	#Warn.	SFDR	#Warn.	SFDR	#Warn.	SFDR	#Warn.	SFDR	#Warn.	SFDR	#Warn.	SFDR
CWE-401	linux/sound	$0^{(0)}$	–	$0^{(0)}$	–	$2^{(2)}$	0.00%	$2^{(2)}$	100.00%	$0^{(0)}$	–	$0^{(0)}$	–	$0^{(0)}$	–
	linux/mm	$0^{(0)}$	–	$0^{(0)}$	–	$6^{(6)}$	33.33%	$2^{(2)}$	100.00%	$0^{(0)}$	–	$0^{(0)}$	–	$0^{(0)}$	–
	ImageMagick	$0^{(0)}$	–	$0^{(0)}$	–	$3^{(3)}$	0.00%	$3^{(3)}$	66.67%	$141^{(141)}$	89.36%	$37^{(37)}$	100.00%	$0^{(0)}$	–
Average of CWE-401		0	–	0	–	3.67	18.18%	2.33	85.71%	47	89.36%	12.33	100.00%	0	–
CWE-416	linux/net	$3^{(3)}$	66.67%	$0^{(0)}$	–	$1^{(1)}$	0.00%	$5^{(5)}$	80.00%	$0^{(0)}$	–	$0^{(0)}$	–	$72^{(72)}$	97.22%
	linux/drivers/net	$1^{(1)}$	100.00%	$48^{(48)}$	81.25%	$3^{(3)}$	0.00%	$9^{(9)}$	100.00%	$1^{(1)}$	100.00%	$0^{(0)}$	–	$74^{(74)}$	100.00%
	vim	$31^{(31)}$	100.00%	$0^{(0)}$	–	$0^{(0)}$	–	$6^{(6)}$	100.00%	$0^{(0)}$	–	$2^{(2)}$	100.00%	$0^{(0)}$	–
Average of CWE-416		11.67	97.14%	16.00	81.25%	1.33	0.00%	6.67	95.00%	0.33	100.00%	0.67	100.00%	48.67	98.63%
CWE-476	linux/drivers/peci	$3^{(3)}$	100.00%	$0^{(0)}$	–	$0^{(0)}$	–	$4^{(4)}$	100.00%	$3^{(3)}$	100.00%	$0^{(0)}$	–	$0^{(0)}$	–
	gpac	$77^{(77)}$	97.40%	$0^{(0)}$	–	$4^{(4)}$	0.00%	$15^{(15)}$	26.67%	$6345^{(300)}$	76.00%	$1^{(1)}$	100.00%	$194^{(194)}$	75.77%
	bitlbee	$374^{(300)}$	91.00%	$0^{(0)}$	–	$3^{(3)}$	0.00%	$8^{(8)}$	87.50%	$284^{(284)}$	94.72%	$0^{(0)}$	–	$5^{(5)}$	80.00%
Average of CWE-476		151.33	92.37%	0	–	2.33	0.00%	9.00	55.56%	2210.67	85.18%	0.33	100.00%	66.33	75.88%
Overall Average		54.33	92.77%	5.33	81.25%	2.44	9.09%	6.00	74.07%	752.67	86.01%	4.44	100.00%	38.33	85.51%

TABLE VIII: Detection results of the evaluated tools using GPT-5.4 on real-world C/C++ projects. In this table, #Warn. denotes the number of warnings, superscripts indicate SFDR sample sizes. – indicates that the SFDR cannot be calculated.

CWE	Project	RepoAudit$_{GPT-5.4}$		KNighter$_{GPT-5.4}$	
		#Warn.	SFDR	#Warn.	SFDR
CWE-401	linux/sound	$0^{(0)}$	–	$0^{(0)}$	–
	linux/mm	$0^{(0)}$	–	$0^{(0)}$	–
	ImageMagick	$0^{(0)}$	–	$0^{(0)}$	–
Average of CWE-401		0	–	0	–
CWE-416	linux/net	$2^{(2)}$	0.00%	$5^{(5)}$	100.00%
	linux/drivers/net	$2^{(2)}$	100.00%	$265^{(265)}$	95.47%
	vim	$14^{(14)}$	100.00%	$0^{(0)}$	–
Average of CWE-416		6	88.89%	90.00	95.56%
CWE-476	linux/drivers/peci	$1^{(1)}$	100.00%	$0^{(0)}$	–
	gpac	$95^{(95)}$	95.79%	$0^{(0)}$	–
	bitlbee	$339^{(300)}$	95.67%	$0^{(0)}$	–
Average of CWE-476		145	95.71%	0	–
Overall Average		50.33	95.41%	30.00	95.56%

Finding 1: General-agent methods achieve the highest recall and precision, but still struggle with complex code. Both LLM-based and traditional tools suffer from incomplete API modeling, and providing project-specific API specifications substantially improves detection performance.

Finding 2: Specialized LLM-based methods rely on rigid analysis workflows, which can limit stronger models' capabilities while making their performance less sensitive to weaker backbones.

B. RQ2: Effectiveness in Real-world Deployment Scenarios

Table VII, VIII and Table IX, X summarize the effectiveness of the evaluated methods on large and latest real-world projects/modules. Since most tool-project/module detection results contain no more than 300 warnings (185 out of 195 experimental setups), for each tool's detection results on each project or module, we sampled up to **300** reports for each tool on each project or module (resulting in **6,442** reports in total) and provided them to Codex with the prompt[3]. Specifically, for each warning, we provided both the warning report and the corresponding repository, enabling Codex to inspect the relevant files and determine whether the warning was a TP or FP. For traditional tools, the report includes checker information, involved code locations, and data-flow information when available. For LLM-based tools, it includes the dataflow and code locations related to the reported vulnerability, as well as the method's original vulnerability analysis. Finally, to validate the reliability of this process, we performed stratified sampling on the labeled results by sampling up to five TPs and five FPs for each tool on each repository. In total, we manually validated 197 TPs and 424 FPs, and the Codex labels achieved an accuracy of 94.36%. We also estimated the population-weighted TP/FP labeling accuracy as 98.16%. We evaluated its precision at the 95% confidence level using a normal approximation based on stratified sampling variance with finite-population correction [68]. **Overall, the results reveal that LLM-based detectors still struggle when applied in real-world scenarios, with high false discovery rates that limit their practical effectiveness.**

For C/C++ projects/modules, specialized LLM-based methods show unsatisfactory performance. RepoAudit$_{Sonnet3.5}$ produces a number of warnings for CWE-476, while the vast majority are FPs, resulting in an average SFDR of 92.37% for CWE-476. KNighter$_{O3-mini}$ produces 48 warnings in `linux/drivers/net`, achieving a high SFDR (81.25%). Traditional tools often generate substantially more warnings, but most of them are false positives. For example, CodeQL reports 6,345 warnings on `gpac` for CWE-476 with an SFDR of 76.00%. Most of these reports (4,827 of them) stem from the coding manners of this project, in which the relevant APIs guarantee non-`NULL` return values under their usage conventions, so the callers omit explicit null checks. Nevertheless, these safe call sites match the ``Returned pointer not checked'' pattern, and are flagged as warnings. CSA reports 72 and 74 warnings on the two Linux CWE-416

[3]https://github.com/Feng-Jay/reports-filter/blob/master/src/codex.rs

TABLE IX: Detection results of evaluated tools on real-world Java projects. In this table, #Warn. denotes the number of warnings, superscripts indicate SFDR sample sizes. – indicates that the tool does not support the corresponding CWE type or that the SFDR cannot be calculated.

CWE	Project	IRIS$_{GPT-4}$		LLMDFA$_{GPT-4}$		Codex$_{GPT-5.4}$		Claude Code$_{Sonnet4.6}$		CodeQL		Semgrep		SpotBugs		INFERROI$_{GPT-4}$	
		#Warn.	SFDR	#Warn.	SFDR	#Warn.	SFDR	#Warn.	SFDR	#Warn.	SFDR	#Warn.	SFDR	#Warn.	SFDR	#Warn.	SFDR
CWE-022	OpenOLAT	**709**$^{(300)}$	89.67%	–	–	1$^{(1)}$	0.00%	2$^{(2)}$	50.00%	63$^{(63)}$	71.43%	183$^{(183)}$	90.71%	677$^{(300)}$	77.33%	–	–
	spark	7$^{(7)}$	42.86%	–	–	3$^{(3)}$	0.00%	5$^{(5)}$	60.00%	5$^{(5)}$	40.00%	5$^{(5)}$	80.00%	8$^{(8)}$	62.50%	–	–
	Dspace	552$^{(300)}$	98.00%	–	–	3$^{(3)}$	66.67%	1$^{(1)}$	0.00%	2$^{(2)}$	100.00%	107$^{(107)}$	96.26%	290$^{(290)}$	94.83%	–	–
Average of CWE-022		422.67	93.25%	–	–	2.33	28.57%	2.67	50.00%	23.33	70.00%	98.33	92.54%	325	85.62%	–	–
CWE-078	xstream	17$^{(17)}$	100.00%	0$^{(0)}$	–	2$^{(2)}$	100.00%	4$^{(4)}$	100.00%	0$^{(0)}$	–	0$^{(0)}$	–	0$^{(0)}$	–	–	–
	workflow-cps-plugin	5$^{(5)}$	100.00%	0$^{(0)}$	–	0$^{(0)}$	–	4$^{(4)}$	100.00%	0$^{(0)}$	–	0$^{(0)}$	–	0$^{(0)}$	–	–	–
	tika	71$^{(71)}$	100.00%	4$^{(4)}$	100.00%	0$^{(0)}$	–	5$^{(5)}$	100.00%	13$^{(13)}$	100.00%	32$^{(32)}$	96.88%	31$^{(31)}$	96.77%	–	–
Average of CWE-078		31	100.00%	1.33	100.00%	0.67	100.00%	4.33	100.00%	4.33	100.00%	10.67	96.88%	10.33	96.77%	–	–
CWE-079	xwiki-platform	**458**$^{(300)}$	99.67%	5$^{(5)}$	100.00%	2$^{(2)}$	0.00%	7$^{(7)}$	85.71%	1$^{(1)}$	100.00%	1$^{(1)}$	100.00%	2$^{(2)}$	100.00%	–	–
	jenkins	208$^{(208)}$	100.00%	4$^{(4)}$	100.00%	2$^{(2)}$	50.00%	11$^{(11)}$	100.00%	0$^{(0)}$	–	1$^{(1)}$	0.00%	0$^{(0)}$	–	–	–
	keycloak	358$^{(300)}$	100.00%	2$^{(2)}$	100.00%	2$^{(2)}$	0.00%	5$^{(5)}$	80.00%	6$^{(6)}$	100.00%	0$^{(0)}$	–	0$^{(0)}$	–	–	–
Average of CWE-079		341.33	99.88%	3.67	100.00%	2.00	16.67%	7.67	91.30%	2.33	100.00%	0.67	50.00%	0.67	100.00%	–	–
CWE-094	onedev	128$^{(128)}$	100.00%	–	–	0$^{(0)}$	–	5$^{(5)}$	100.00%	0$^{(0)}$	–	0$^{(0)}$	–	0$^{(0)}$	–	–	–
	activemq	271$^{(271)}$	100.00%	–	–	0$^{(0)}$	–	6$^{(6)}$	100.00%	1$^{(1)}$	100.00%	0$^{(0)}$	–	0$^{(0)}$	–	–	–
	cron-utils	5$^{(5)}$	100.00%	–	–	1$^{(1)}$	100.00%	1$^{(1)}$	100.00%	0$^{(0)}$	–	0$^{(0)}$	–	0$^{(0)}$	–	–	–
Average of CWE-094		134.67	100.00%	–	–	0.33	100.00%	4.00	100.00%	0.33	100.00%	0	–	0	–	–	–
CWE-772	sql2o	–	–	–	–	3$^{(3)}$	0.00%	5$^{(5)}$	80.00%	0$^{(0)}$	–	–	–	0$^{(0)}$	–	23$^{(23)}$	95.65%
	RxJava	–	–	–	–	2$^{(2)}$	50.00%	10$^{(10)}$	60.00%	0$^{(0)}$	–	–	–	0$^{(0)}$	–	890$^{(300)}$	98.00%
	jsoup	–	–	–	–	1$^{(1)}$	0.00%	6$^{(6)}$	50.00%	9$^{(9)}$	77.78%	–	–	0$^{(0)}$	–	66$^{(66)}$	89.39%
Average of CWE-772		–	–	–	–	2.00	16.67%	7.00	61.90%	3	77.78%	–	–	0.00	–	326.33	96.40%
Average		232.42	97.80%	2.50	100.00%	1.47	31.82%	5.13	81.82%	6.67	77.00%	27.42	92.71%	67.20	86.21%	326.33	96.40%

modules, both with SFDRs above 97%. Similarly, Semgrep emits fewer warnings overall, but its warnings are frequently unreliable, reaching 100.00% SFDR on several projects. Moreover, most methods still suffer from the incomplete API modeling issue. RepoAudit$_{Sonnet3.5}$ reports no CWE-401 warnings, while KNighter$_{O3-mini}$ reports warnings only in `linux/drivers/net`. CodeQL, Semgrep, and CSA also produce no warnings on many projects or modules.

In contrast, the general-agent methods produce far fewer warnings. Codex is the most conservative, with only 2.44 warnings on average and the lowest overall SFDR of 9.09%, suggesting that it avoids large-scale warning floods. Claude Code reports more warnings than Codex and detects more candidate issues, but its overall SFDR is much higher at 74.07%. Our analysis shows that this is because Claude Code tends to report suspicious code locations more aggressively, often before fully validating the related data structure or feasible data-flow path. Which is further explained in Section V-C.

For Java projects, a similar pattern emerges. IRIS$_{GPT-4}$ consistently produces the largest number of warnings among all evaluated tools, yet our inspection again reveals almost no true positives, with an average SFDR of **97.80%**. In contrast, LLMDFA$_{GPT-4}$ generates only a small number of reports, and all of them are identified as FPs, yielding an SFDR of 100.00%. Similarly, INFERROI$_{GPT-4}$ reports many warnings for CWE-772, with an average SFDR of 96.40%. Traditional tools also suffer from high SFDRs. Semgrep and SpotBugs reach 92.71% and 86.21%, respectively, while CodeQL has a lower overall SFDR of 77.00%. And due to the predefined rule sets of these tools, they can generate warnings in only about half of the selected projects. In contrast, general-agent methods produce substantially fewer warnings. Codex is the most conservative, reporting only 1.47 warnings on average and achieving the lowest overall SFDR of 31.82%, whereas Claude Code reports more candidate issues, with 5.13 warnings on average, but has a higher SFDR of 81.82%.

TABLE X: Detection results of the evaluated tools using GPT-5.4 on real-world Java projects. In this table, #Warn. denotes the number of warnings, superscripts indicate SFDR sample sizes. – indicates that the SFDR cannot be calculated.

CWE	Project	IRIS$_{GPT-5.4}$		LLMDFA$_{GPT-5.4}$		INFERROI$_{GPT-5.4}$	
		#Warn.	SFDR	#Warn.	SFDR	#Warn.	SFDR
CWE-022	OpenOLAT	131$^{(131)}$	68.70%	–	–	–	–
	spark	11$^{(11)}$	0.00%	–	–	–	–
	Dspace	19$^{(19)}$	73.68%	–	–	–	–
Average of CWE-022		53.67	64.60%	–	–	–	–
CWE-078	xstream	0$^{(0)}$	–	0$^{(0)}$	–	–	–
	workflow-cps-plugin	0$^{(0)}$	–	0$^{(0)}$	–	–	–
	tika	12$^{(12)}$	100.00%	1$^{(1)}$	100.00%	–	–
Average of CWE-078		4	100.00%	0.33	100.00%	–	–
CWE-079	xwiki-platform	26$^{(26)}$	100.00%	3$^{(3)}$	100.00%	–	–
	jenkins	14$^{(14)}$	100.00%	4$^{(4)}$	100.00%	–	–
	keycloak	7$^{(7)}$	100.00%	2$^{(2)}$	100.00%	–	–
Average of CWE-079		15.67	100.00%	3.00	100.00%	–	–
CWE-094	onedev	1$^{(1)}$	100.00%	–	–	–	–
	activemq	47$^{(47)}$	100.00%	–	–	–	–
	cron-utils	0$^{(0)}$	–	–	–	–	–
Average of CWE-094		16.00	100.00%	–	–	–	–
CWE-772	sql2o	–	–	–	–	17$^{(17)}$	94.12%
	RxJava	–	–	–	–	1016$^{(300)}$	98.00%
	jsoup	–	–	–	–	35$^{(35)}$	80.00%
Average of CWE-772		–	–	–	–	356.00	96.02%
Overall Average		22.33	78.73%	1.67	100.00%	356.00	96.02%

Collectively, these results indicate that both current LLM-based and traditional static vulnerability detectors struggle to be directly used in real-world scenarios, typically failing either due to near-zero recall, as explained in Section V-A, or overwhelming numbers of false positives in our setting. These observations motivate our deeper investigation in RQ3, where we examine the underlying causes of these false positives and analyze why existing techniques break down.

A. Limited Context & Dataflow Construction

- **A1** **Truncated Interprocedural Context:** fails to analyze project-level dataflow across functions, files, or modules.
- **A2** **Incorrect control flow:** constructs or follows structurally unreachable control-flow paths, e.g., by combining mutually exclusive branches.

B. Misidentification of vulnerability-relevant program points.

- **B1** **Incorrect program-point classification:** classifies benign program points as vulnerability-relevant.

C. Prompt Misalignment

- **C1** **Prompt deviations:** fails to follow the specified prompt constraints or the target CWE definition.

D. Code Semantics Misinterpretation

- **D1** **Missed key program points:** overlooks or fails to model guards, sanitizers, initialization, or resource-release operations.
- **D2** **Poor language-feature support:** misunderstands language-specific constructs such as constructors, destructors, exceptions, or try-with-resources.
- **D3** **Path-insensitive analysis:** reports flows that are infeasible under branch or value constraints.
- **D4** **Type/structure blindness:** ignores type, class, field, or data-structure constraints needed to judge feasibility.

Fig. 7: Taxonomy of FP reasons that are related to the cases detected by the selected methods.

Finding 3: On real-world projects, specialized LLM-based and traditional tools suffer from high SFDRs and limited coverage, while general agents reduce warning volume but still face false-positive risks.

C. RQ3: Causes of False Positives

As described in Section III-D, we first derived a taxonomy of false-positive causes from manual inspection. Two authors with a background in security and software engineering manually analyzed the sampled false positives. We applied an inductive open-coding procedure with double coding [69]. Each author first reviewed all examples to derive a coherent taxonomy of FP causes and iteratively refined the codebook. After consolidating and resolving disagreements on category definitions, the experts independently assigned the primary FP reason for each case. For each warning, our goal is to identify the most direct cause of the false positive. The annotation follows a hierarchical decision procedure, assigning each case to the first applicable category in the following order:

We first examine whether the selected program points are correctly classified as vulnerability-relevant (Category B, shown in Fig. 7). If they are correct, we then inspect the reported code region or path to determine whether the analysis relies on incomplete or theoretically invalid data/control-flow reasoning (Category A). If the analyzed path or context is reasonable, we further check whether the tool misinterprets program semantics when drawing the vulnerability conclusion (Category D). Finally, for LLM-based tools, we examine whether the final judgment deviates from the prompt or target CWE definition (Category C). This ordering reflects dependencies in the report's justification: flow analysis is assessed relative to the selected program points, and subsequent semantic judgments depend on adequate context and a valid control-flow structure. An earlier failure must therefore be corrected before later reasoning can validate the report. For example, semantic reasoning cannot justify a vulnerability claim based on a structurally invalid path; we therefore assign A as the primary cause, even when semantic errors (D) also occur. Finally, they reconciled disagreements and consolidated the labels into a unified annotation set. We then used Codex with the prompt[4] to label the **5,896** false positives generated from Section V-B according to this taxonomy, while allowing it to propose a new reason if none of the existing categories fit. In the prompt, for each warning from each tool, we provided the agent with the tool's workflow description, the taxonomy and category definitions derived from the previous manual analysis, the corresponding repository code, and the warning information produced by the tool. The warning information typically includes the vulnerability type, reported location, and data-flow/path information when available. And for LLM-based tools, it also includes the LLM-generated explanation or analysis. After manually reviewing and consolidating the labels produced by Codex, we found that no additional FP category was needed, and the final annotations followed the established taxonomy. To assess labeling reliability, we further performed stratified sampling on the Codex-generated labels for manual validation. Specifically, for each tool and each FP reason, we sampled up to five cases separately for C/C++ and Java, resulting in 425 validation cases in total. The Codex-generated labels achieved an overall accuracy of 87.76%, and 77 out of 91 tool-reason groups achieved an accuracy of at least 80%. Most misclassifications involved explanations with multiple contributing factors, where Codex did not consistently apply the criteria for selecting the primary cause. We also estimated the population weighted FP-reason classification accuracy as 92.52%. We evaluated its precision at the 95% confidence level using a normal approximation based on stratified sampling variance with finite-population correction [68].

In the following, we analyze several representative reasons from this taxonomy. The taxonomy and the definition of each reason are summarized in Fig. 7 and Table XI. Due to page limits, we present three representative illustrative examples here, and include more detailed examples for the remaining FP reasons in Appendix A. As shown in the figures, LLM-based and traditional methods share two major sources of false positives: **(1) Truncated Interprocedural Context (Reason A1).** To limit analysis complexity, most tools rely either on source–sink style reasoning or on fixed-depth call-graph exploration. As shown in Fig. 8, Claude Code reports a memory-leak warning because it ob-

[4]https://github.com/Feng-Jay/reports-filter/blob/master/src/codex.rs

TABLE XI: Distribution of false-positive reasons by tool. Each cell reports the reason count over the total number of false positives for that tool, followed by the corresponding percentage. Color intensity is rank-scaled independently within each tool row.

Tool	A1	A2	B1	C1	D1	D2	D3	D4
RepoAudit$_{Sonnet3.5}$	34 / 385 (8.83%)	58 / 385 (15.06%)	226 / 385 (58.70%)	0 / 385 (0.00%)	53 / 385 (13.77%)	0 / 385 (0.00%)	12 / 385 (3.12%)	2 / 385 (0.52%)
RepoAudit$_{GPT-5.4}$	36 / 395 (9.11%)	31 / 395 (7.85%)	296 / 395 (74.94%)	0 / 395 (0.00%)	14 / 395 (3.54%)	0 / 395 (0.00%)	13 / 395 (3.29%)	5 / 395 (1.27%)
KNighter$_{O3-mini}$	0 / 39 (0.00%)	4 / 39 (10.26%)	31 / 39 (79.49%)	0 / 39 (0.00%)	0 / 39 (0.00%)	0 / 39 (0.00%)	2 / 39 (5.13%)	2 / 39 (5.13%)
KNighter$_{GPT-5.4}$	2 / 258 (0.78%)	22 / 258 (8.53%)	204 / 258 (79.07%)	0 / 258 (0.00%)	2 / 258 (0.78%)	0 / 258 (0.00%)	13 / 258 (5.04%)	15 / 258 (5.81%)
IRIS$_{GPT-4}$	96 / 1870 (5.13%)	36 / 1870 (1.93%)	1640 / 1870 (87.70%)	3 / 1870 (0.16%)	80 / 1870 (4.28%)	1 / 1870 (0.05%)	4 / 1870 (0.21%)	10 / 1870 (0.53%)
IRIS$_{GPT-5.4}$	11 / 211 (5.21%)	17 / 211 (8.06%)	116 / 211 (54.98%)	0 / 211 (0.00%)	45 / 211 (21.33%)	4 / 211 (1.90%)	2 / 211 (0.95%)	16 / 211 (7.58%)
INFERROI$_{GPT-4}$	57 / 375 (15.20%)	1 / 375 (0.27%)	248 / 375 (66.13%)	0 / 375 (0.00%)	65 / 375 (17.33%)	3 / 375 (0.80%)	0 / 375 (0.00%)	1 / 375 (0.27%)
INFERROI$_{GPT-5.4}$	74 / 338 (21.89%)	2 / 338 (0.59%)	181 / 338 (53.55%)	0 / 338 (0.00%)	76 / 338 (22.49%)	2 / 338 (0.59%)	1 / 338 (0.30%)	2 / 338 (0.59%)
LLMDFA$_{GPT-4}$	0 / 15 (0.00%)	0 / 15 (0.00%)	12 / 15 (80.00%)	0 / 15 (0.00%)	3 / 15 (20.00%)	0 / 15 (0.00%)	0 / 15 (0.00%)	0 / 15 (0.00%)
LLMDFA$_{GPT-5.4}$	0 / 10 (0.00%)	0 / 10 (0.00%)	8 / 10 (80.00%)	0 / 10 (0.00%)	2 / 10 (20.00%)	0 / 10 (0.00%)	0 / 10 (0.00%)	0 / 10 (0.00%)
Codex$_{GPT-5.4}$	0 / 9 (0.00%)	2 / 9 (22.22%)	3 / 9 (33.33%)	1 / 9 (11.11%)	3 / 9 (33.33%)	0 / 9 (0.00%)	0 / 9 (0.00%)	0 / 9 (0.00%)
Claude Code$_{Sonnet4.6}$	20 / 103 (19.42%)	15 / 103 (14.56%)	43 / 103 (41.75%)	0 / 103 (0.00%)	19 / 103 (18.45%)	1 / 103 (0.97%)	3 / 103 (2.91%)	2 / 103 (1.94%)
CSA	87 / 295 (29.49%)	54 / 295 (18.31%)	86 / 295 (29.15%)	0 / 295 (0.00%)	11 / 295 (3.73%)	0 / 295 (0.00%)	47 / 295 (15.93%)	10 / 295 (3.39%)
CodeQL	252 / 704 (35.80%)	110 / 704 (15.62%)	114 / 704 (16.19%)	0 / 704 (0.00%)	66 / 704 (9.38%)	15 / 704 (2.13%)	142 / 704 (20.17%)	5 / 704 (0.71%)
Semgrep	93 / 345 (26.96%)	17 / 345 (4.93%)	170 / 345 (49.28%)	0 / 345 (0.00%)	57 / 345 (16.52%)	3 / 345 (0.87%)	0 / 345 (0.00%)	5 / 345 (1.45%)
SpotBugs	99 / 544 (18.20%)	32 / 544 (5.88%)	356 / 544 (65.44%)	0 / 544 (0.00%)	47 / 544 (8.64%)	1 / 544 (0.18%)	8 / 544 (1.47%)	1 / 544 (0.18%)

serves that `create_cache` may return at Line 15 without directly releasing the allocated `pool->handle_cachep` from Line 8. However, this is a false positive. Claude Code fails to recognize that `pool` is passed by pointer and that ownership is handled by the caller. When `zs_create_pool` calls `create_cache` and the call fails, it jumps to the error-handling path, invokes `zs_destroy_pool`, and eventually releases `pool->handle_cachep` through `destroy_cache` via `kmem_cache_destroy`. Thus, the warning is caused by incomplete interprocedural reasoning over caller-side cleanup logic. Similarly, data-flow analyses in traditional tools such as CodeQL track dataflows between configured source and sink points. However, in real-world projects, where data may originate from diverse entry points and propagate through rich, multi-hop control-data paths, such restricted flow modeling often fails to capture the full chain from input to output, leading to extensive false positives. **(2) Incorrect program-point classification (Reason B1)**, where tools either misclassify benign program points as security-relevant or fail to correctly interpret the semantics of API usages. As shown in Fig. 9, the LLM misidentifies the variable `new` as a pointer to newly allocated memory in C++ and assumes it may be `NULL`. Even if it were uninitialized, using it would not constitute an NPD. At most, it would be an uninitialized-value issue. Therefore, the warning results from incorrect program-point classification (B1). However, throughout the entire reasoning process on this path, the LLM fails to recognize this and continues to treat the use of `new` as a potentially NPD vulnerability at line 17.

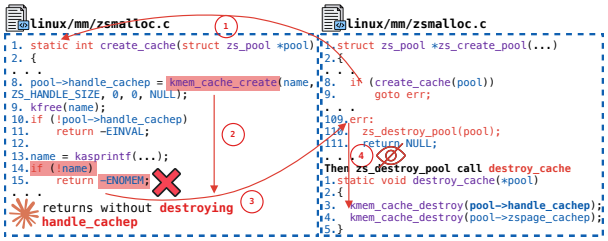


Fig. 8: An illustrating FP example of reason A1

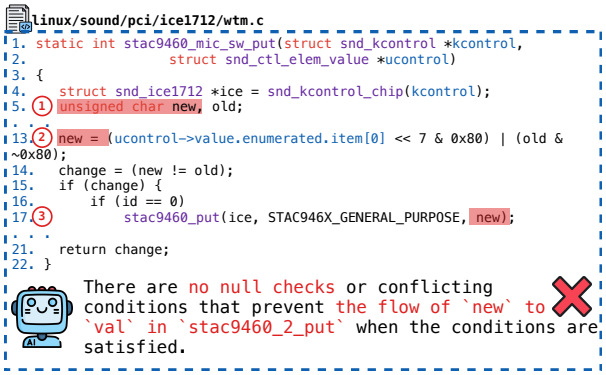


Fig. 9: An illustrating FP example of reason B1

Beyond the shared sources of false positives, we further analyze the reasons that are particularly prevalent in LLM-based vulnerability detection methods. Although LLMs exhibit strong code analysis abilities, their performance on vulnera-

bility detection still faces several inherent limitations: **First, LLMs often fail to recover the correct execution flow in the presence of complex control structures (Reason A2).** For example, 15.06% of RepoAudit$_{Sonnet3.5}$'s false positives arise because it overlooks key control-flow branches or constructs an incorrect data-flow graph, causing the detector to analyze paths that do not actually exist in the program. There is another interesting example in Fig. 10, in the `vim` project, there are two `gui_mch_destroy_scrollbar` functions with the same signature, defined in `gui_haiku.cc` and `gui_w32.c` to support different operating systems. In practice, these two functions cannot be invoked together in a single build. However, when the LLM analyzes the two consecutive invocations of `gui_mch_destroy_scrollbar` in `window.c`, it incorrectly concludes that the first call resolves to the implementation in `gui_haiku.cc` and the second to the implementation in `gui_w32.c`, merging mutually exclusive execution paths and leading to an incorrect understanding of the program behavior. **Second, LLMs frequently overlook implicit sanitization logic within a dataflow (Reason D1).** Most selected LLM-based methods suffer noticeably from this issue, with 7 out of 12 settings exceeding 15.00%, leading them to flag dataflows as vulnerable even when proper sanitizers exist along the path. **Finally, LLMs sometimes do not follow the prompt or CWE definition well (Reason C1).** Both Codex and IRIS flag benign code as vulnerable or report issues that do not conform to the CWE definitions specified in the prompt. The corresponding examples are shown in our Appendix A.

Furthermore, for specialized LLM-based methods, the impact of stronger models differs depending on where the LLM is used in the detection workflow. For methods where the LLM directly participates in vulnerability judgment, such as RepoAudit and KNighter, where vulnerable paths or patterns are analyzed by LLMs, the proportions of Category A and D errors decrease, while program-point classification errors increase substantially. In contrast, for methods where LLMs are mainly used for API-role inference and the final detection is performed by a more constrained analysis procedure, such as IRIS with CodeQL and INFERROI over CFGs, program-point classification errors decrease more clearly. This suggests that stronger models can reduce errors in the parts of the workflow they directly improve, but the overall effectiveness remains constrained by each method's workflow design and validation mechanism.

For RepoAudit, KNighter, and CSA, which rely on either LLMs or static path-enumeration mechanisms to extract path information before applying fixed vulnerability-checking logic, the proportion of A2 errors is much higher. This suggests current path extraction results remain unreliable under complex control flow, and errors in the extracted path can directly propagate to the final vulnerability judgment. For general-agent methods, our session analysis of Codex and Claude Code shows that, across 24 real-world projects/modules, Claude Code invoked tools 2,702 times and reported 131 warnings, while Codex invoked tools 1,415 times and reported 44 warnings. On average, this corresponds to 20.63 tool invocations per reported warning for Claude Code and 32.16 for Codex.

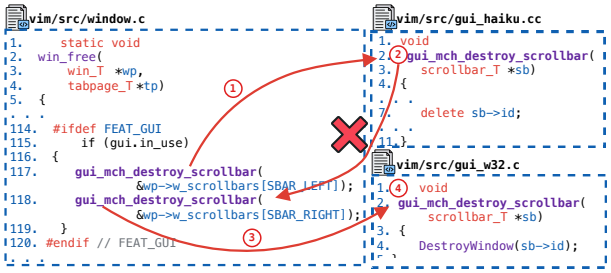


Fig. 10: An illustrating FP example of reason A2

This suggests that Codex spends more analysis effort on each reported warning, which helps explain its much lower SFDR and the lower frequency of analysis-related FP reasons compared with Claude Code.

These findings suggest that although LLMs are capable of reasoning about code semantics, reliably capturing precise and complete program behavior in complex real-world projects, as well as strictly adhering to prompt specifications, remain major challenges.

> **Finding 4:** False positives are dominated by truncated interprocedural context (A1), incorrect program-point classification (B1), which account for 14.60% (861 / 5,896) and 63.33% (3,734 / 5,896) of the total FPs, respectively. LLM-based tools further incur errors due to hallucinated code understanding/reasoning, and imperfect adherence to prompts (Reason A2, C1, D1).

D. RQ4: Detection Overhead at Project Scale

One primary concern for LLM-based methods is their cost and efficiency. In this research question, we measure the token consumption and time cost of the selected tools. Table XII reports the average overhead of these methods, Fig. 11 further reports their warning-level cost, and more detailed statistics are provided in Appendix C. Overall, compared to traditional tools, LLM-based vulnerability detection methods impose **substantial computational costs, raising practical concerns for project-scale adoption.** Specifically, among specialized LLM-based methods, multi-agent methods such as RepoAudit and LLMDFA exhibit the highest computational overhead on C/C++ and Java projects. RepoAudit can consume more than **225 million** input tokens and more than **38 million** output tokens for a single project, resulting in an average cost of $172.77 and a maximum analysis time of over **800 minutes** (more than 13 hours) per project. Similarly, LLMDFA also incurs extremely high token consumption, up to **38 million** input tokens, and requires $61.60 and **2000 minutes** (more than 33 hours) on average to analyze a single Java project. These results indicate that current LLM-based approaches face a severe scalability challenge when applied to large real-world repositories. At the warning level, LLMDFA remains the most expensive method, requiring more than **$24** and over **800 minutes** per warning under both model settings. These results indicate that current LLM-based approaches face a

TABLE XII: Computational overhead of evaluated tools on real-world projects. – represents tools that do not rely on LLMs.

Category	Method	Input Tokens (K)			Output Tokens (K)			Financial ($)			Time (min)		
		Min	Max	Avg	Min	Max	Avg	Min	Max	Avg	Min	Max	Avg
Specialized LLM-based	RepoAudit$_{*Sonnet3.5}$	0.00	225,493.97	32,634.78	0.00	38,078.26	4,991.27	0.00	1,247.66	172.77	3.32	312.87	65.40
	RepoAudit$_{@GPT-5.4}$	0.00	143,791.47	21,118.02	0.00	13,799.98	2,157.24	0.00	566.48	85.15	2.77	894.63	196.71
	KNighter$_{@O3-mini}$	61.41	185.25	124.73	16.48	47.92	36.55	0.14	0.41	0.34	37.55	221.51	144.09
	KNighter$_{@GPT-5.4}$	103.31	201.95	165.71	27.39	373.73	151.72	0.67	6.09	2.36	35.20	232.57	104.57
	IRIS$_{@GPT-4}$	8.47	1,752.77	508.77	1.45	267.13	103.60	0.04	8.39	2.59	2.55	2,089.47	213.44
	IRIS$_{@GPT-5.4}$	19.40	1,011.07	310.73	3.17	274.94	83.15	0.10	5.82	2.02	4.72	122.38	35.69
	LLMDFA$_{@GPT-4}$	2,535.12	38,310.33	17,445.71	69.50	2,646.89	1,199.05	7.91	135.48	61.60	44.00	4,638.00	2,000.00
	LLMDFA$_{@GPT-5.4}$	2,565.84	28,561.13	16,269.34	52.87	2,600.17	1,174.38	9.54	110.41	58.29	57.58	3,460.48	1,504.01
	INFERROI$_{@GPT-4}$	142.50	2,871.49	1,193.45	64.51	1,408.23	575.84	1.32	28.30	11.62	20.25	176.27	78.10
	INFERROI$_{@GPT-5.4}$	153.61	2,933.26	1,230.17	82.95	1,565.81	660.73	1.63	30.82	12.99	12.57	97.14	43.74
General-Agents	Codex$_{@GPT-5.4}$	359.25	5,999.23	1,958.57	13.26	79.99	32.39	1.21	16.20	5.38	5.22	32.70	14.80
	Claude Code$_{*Sonnet4.6}$	72.41	14,620.30	3,043.48	3.56	127.44	36.18	0.40	45.48	9.67	1.94	49.54	13.19
Traditional	CodeQL	–	–	–	–	–	–	–	–	–	0.21	378.95	21.29
	Semgrep	–	–	–	–	–	–	–	–	–	0.07	2.43	0.64
	CSA	–	–	–	–	–	–	–	–	–	0.25	123.05	60.92
	SpotBugs	–	–	–	–	–	–	–	–	–	0.72	10.90	3.11

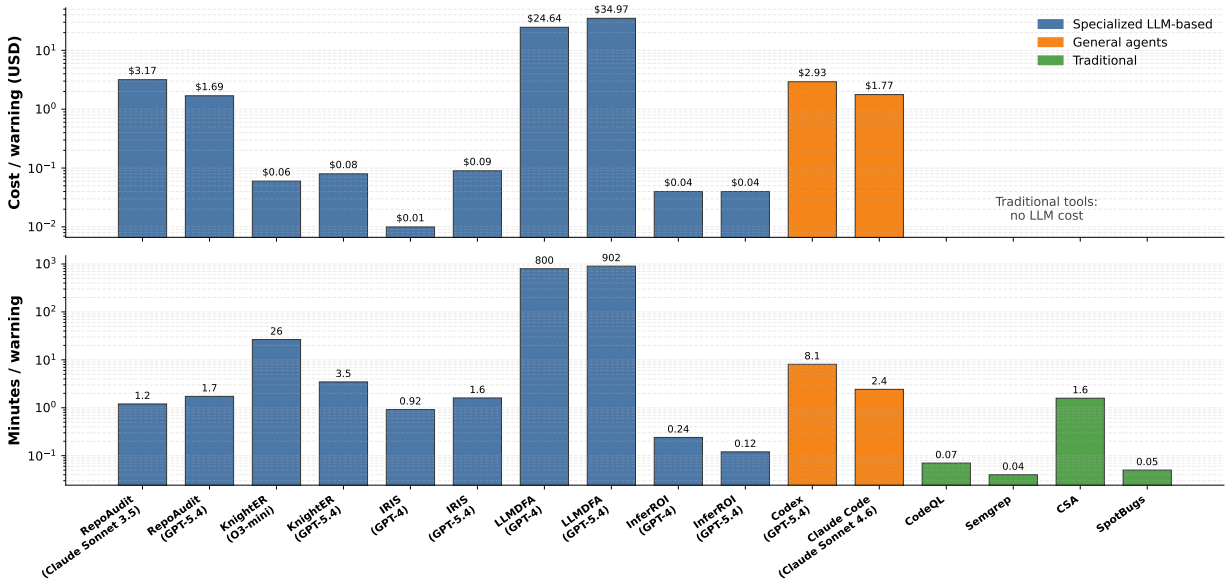


Fig. 11: Financial and time cost per warning for each tool on real-world projects/modules.

severe scalability challenge when applied to large real-world repositories/modules.

Hybrid approaches such as IRIS and INFERROI, which integrate LLMs with static analysis tools or methods, show more moderate overhead, though still requiring hundreds of thousands to millions of tokens, incur non-trivial financial costs, and can take multiple hours on large projects. General-agent methods also incur visible warning-level overhead: Codex costs **$2.93** and **8.1 minutes** per warning, while Claude Code costs **$1.77** and **2.4 minutes** per warning. These results indicate that current LLM-based vulnerability detection methods remain computationally expensive, and their scalability is highly sensitive to prompting strategies and detection workflows, posing practical challenges for project scale adoption. Backbone selection offers another opportunity to reduce costs. For example, lower-priced models such as DeepSeek V4 Flash [70] could reduce API charges, but their overall cost-effectiveness requires evaluating token consumption, runtime, and detection effectiveness together.

In contrast, traditional tools (CodeQL, Semgrep, CSA, and SpotBugs) incur negligible overhead, completing analysis within minutes to hours. This difference emphasizes a significant scalability gap: while LLM-based methods provide stronger semantic reasoning capabilities, they do so at the cost of significantly higher computational resources. These results suggest that improving cost efficiency, through better context pruning, incremental analysis, or lighter-weight LLM pipelines, remains essential before LLM-based vulnerability detectors can be deployed reliably at the project scale.

Finding 5: Project-scale LLM-based vulnerability detection is computationally expensive and time-consuming. A single project can require up to hundreds of millions of tokens and a multi-day running time, making scalability a critical bottleneck.

VI. Discussion

A. Implications

Our comprehensive analysis across five specialized LLM-based methods, two general agents, and four traditional tools

reveals that, although LLMs offer promising semantic reasoning capabilities and the potential to generalize beyond manually modeled security patterns, they remain far from mature enough for reliable vulnerability analysis in real-world scenarios. Despite demonstrating an ability to infer higher-level intentions that traditional static tools often miss, current specialized LLM-based methods suffer from several practical limitations that restrict their effectiveness:

(1) Workflow choices create distinct trade-offs.

Our results show that different LLM-based workflows exhibit different strengths and limitations. Specialized methods decompose detection into structured subtasks, such as source/sink inference, path extraction, checker generation, or path validation. However, even with the same backbone model, they may underperform general-agent methods, suggesting that their task formulation can constrain how fully the model's reasoning capability is used. For example, RepoAudit mainly uses the LLM to extract path information and then applies fixed logic for vulnerability judgment, so path-extraction errors directly affect the final result. General-agent methods inspect project context more flexibly and achieve stronger recall, but they still struggle with complex files and large code contexts. Future detectors should therefore combine the flexibility of general agents with specialized program-analysis support, such as explicit path constraints, intermediate representations, and validation modules that check and revise LLM-generated paths or judgments.

(2) Dataflow reasoning is shallow and incomplete.

Both general agent-based and specialized LLM-based methods typically follow a source-first detection workflow: they first search for potential source program points and then initiate subsequent analysis from these identified locations. Consequently, risk is often inferred from local evidence without the global execution context, causing models to overlook sanitization or correct sink behavior that occurs further along the program path. Shallow dataflow is therefore a fundamental bottleneck: when the full path from "input $\rightarrow$ propagation $\rightarrow$ output" is not analyzed holistically, real vulnerabilities may be missed, while safe code is frequently misclassified as risky. One direct mitigation is to add a post-processing stage that further validates generated warnings, as in approaches such as LLM4PFA [71] and BugLens [72]. Such methods can eliminate some false positives, but they mostly focus on branch-condition feasibility and path reachability, while providing limited modeling of program values during execution. A more promising direction is to integrate LLMs with formal analysis during detection. For example, LLMs could be used to infer constraints over program value spaces, while symbolic execution or other formal techniques check vulnerability reachability under global execution context.

(3) Incomplete or incorrect API modeling. Based on our experiments, a considerable portion of FNs and FPs is due to the incomplete and incorrect modeling of project-specific APIs. In real-world projects, vulnerability-relevant operations may be hidden behind wrappers, callbacks, reflection, or framework-managed lifecycles. Although several LLM-based approaches (IRIS, LLMDFA) attempt to infer project-specific or vulnerability-specific sources/sinks via few-shot prompting, the inferred results are often unreliable in practice (e.g., IRIS mislabels `Map.put()` as a sink in Java projects, resulting in an amount of FPs). Thus, robust and context-aware API-role inference remains an unresolved challenge. Future work should move from few-shot API-role prediction toward evidence-grounded specification inference. One direction is to combine LLMs with program-analysis signals, data-dependence relations, API documentation, and usage patterns, to infer vulnerability-relevant API roles and usage requirements. For example, SpecAuditor [73] extracts audit specifications from historical patches and extends them to semantically related entities using documentation. Another direction is to introduce iterative validation: inferred API specifications should be checked against observed dataflows, known sanitizers, and project-specific API contracts before being used for detection. For example, agentic property-based testing [74] infers properties from code and documentation, generates executable tests, and analyzes test failures to assess potential bugs. Building reusable project-level API summaries could further reduce repeated inference errors across related modules and versions.

(4) Semantic understanding in complex contexts remains limited. As shown in Section V-C, a substantial portion of false positives produced by LLM-based vulnerability detectors stems from the misinterpretation of code semantics. In many cases, models overlook key program points along a dataflow (e.g., a sanitizer, a deferred release, a null-check, or a conditional exit), or fail to infer the actual execution behavior of code due to language-specific constructs such as try-with-resources, destructors, template expansion, or macro-controlled branching. These semantic gaps often cause LLMs to reason along incorrect control paths or assume unsafe behavior where none exists, ultimately misclassifying benign code as vulnerable. We suggest future work strengthen semantic reasoning through hybrid workflows, augmenting LLMs with intermediate program representations, graph-based context retrieval, and static analysis signals to provide more complete execution context [16], [75]. In particular, detectors should explicitly model key semantic events, such as ownership transfer, sanitizer effects, error-handling paths, and resource lifecycle operations, rather than relying only on natural-language reasoning over code snippets.

(5) Prompt-alignment and task compliance are inconsistent. In our evaluation, we observe that methods such as Codex and IRIS, which rely on LLMs to perform reasoning over extracted dataflows, may still fail to adhere to the intended vulnerability definition or CWE scope in the prompt. Even when the reasoning is correct at a syntactic or control-flow level, the model occasionally drifts from the task objective, producing conclusions that do not align with the vulnerability type under detection—reflecting semantic drift and task-misalignment. To improve this, future systems may require structured multi-agent collaboration, self-verification modules, or constraint-guided reasoning to enforce consistent detection goals [76], [77].

(6) Scalability remains a bottleneck. Project-scale analysis demands enormous computational overhead. As shown in Section V-D, RepoAudit and LLMDFA consume up to

hundreds of millions of tokens and require hours to days to complete a single project, making them unsuitable for continuous integration. This observation aligns with both the authors' discussions in their papers and the feedback we received from them, who confirmed that the current implementations may incur substantial computational overhead when analyzing large projects. Hybrid approaches (IRIS, INFERROI) reduce, but do not eliminate, the high cost. Future work should prioritize cost-aware detection designs. Promising directions include hierarchical analysis, caching and reusing intermediate reasoning results, incremental analysis across code revisions, and deduplicating similar path queries to avoid redundant LLM calls [75]. Another direction is adaptive invocation, where expensive LLM reasoning is reserved for uncertain or high-risk candidates, while simpler cases are handled by lightweight static checks. Alongside these workflow optimizations, adopting LLMs with efficient architectures and lower inference costs, such as DeepSeek V4 Flash, could further reduce monetary costs. Their effects on token consumption, runtime, and detection effectiveness require further evaluation.

B. Threats to Validity

Internal Validity. Our controlled evaluation focuses on labeled vulnerabilities, so warnings about other genuine vulnerabilities may be counted as FPs. Due to the substantial evaluation cost, we ran each LLM-based configuration once. Although some tools' performance (e.g., IRIS and INFERROI) is close to that reported in their original studies, LLM nondeterminism may still affect our results. Our analysis involves expert judgment. Although two independent authors conducted labeling and reconciliation to reduce subjectivity (Section V-B and V-C), classification accuracy may still be influenced by personal experience, interpretation of code semantics, or reasoning about exploitability. Misreading or misinterpretation of complex code flows may introduce bias into the taxonomy of FP causes. Moreover, our large-scale warning classification relies partly on Codex-generated labels. Although stratified manual validation shows high accuracy, these labels may still contain errors or model biases. To overcome these threats, the two authors adopt an inductive open-coding procedure with double coding [69], independently validated sampled cases, and resolved any disagreements through discussion. We also released all experimental artifacts, including evaluation scripts, prompts, taxonomy labels, and detailed statistics for external inspection and replication.

Another concern is about data-leakage. Pretraining contamination remains possible for both datasets used in this paper. For the controlled dataset, LLMs may have encountered public vulnerability descriptions, patches, or discussions during training. For the real-world dataset, 21 of the 24 project/-module entries use commits postdating the reported August 2025 knowledge cutoffs of GPT-5.4 [62] and Claude Sonnet 4.6 [63]. However, these versions may retain older code, and knowledge cutoffs do not guarantee exclusion from training. Although pretraining contamination cannot be ruled out, we manually inspected the LLM sessions and found no evidence of explicit reliance on CVE records, vulnerability descriptions, patches, or other ground-truth information during detection.

External Validity. Our evaluation considers 5 specialized LLM-based methods, 2 general agents, and 4 traditional vulnerability detectors, selected because they are open-source/publicly available, runnable at the project scale, and representative of emerging design workflows. However, they may not fully represent all possible architectures, languages, or prompting paradigms. As the field evolves rapidly, future models or customized enterprise-scale deployments may demonstrate different capabilities. Moreover, our evaluation covers a set of real-world repositories and modules across two major programming languages (Java, C/C++). Although our evaluation includes 265 known vulnerabilities and 24 real-world projects/modules, it may not capture the full diversity of software systems and programming patterns. Some C/C++ modules are Linux subsystems from the same repository and commit. While they differ in APIs, functionality, and vulnerability patterns, their shared development context may reduce the diversity represented in our evaluation, potentially limiting external validity. Moreover, the eight evaluated CWE types represent only part of the broader vulnerability landscape. Other vulnerability classes, including concurrency vulnerabilities and type confusion, are outside our scope and may require different control-flow, data-flow, or semantic analyses. Our findings may not generalize uniformly to them. Finally, our findings may not generalize uniformly to systems written in Rust, Go, TypeScript, or large multi-language projects. Future work should broaden the evaluation across these settings, adapting detection methods where necessary.

VII. Conclusion

In this paper, we conduct the first comprehensive study of specialized LLM-based vulnerability detectors at the project scale, evaluating five specialized LLM-based methods, two general-purpose agents, and 4 traditional static analysis tools. Our evaluation covers 265 known vulnerabilities across eight CWE types and 24 open-source projects/modules. On the controlled benchmark, general-purpose agents achieve the highest recall, whereas specialized LLM-based detectors exhibit limited recall but provide partial complementary coverage. Our API-guided experiment in Section V-A further confirms that incomplete API modeling is a key factor limiting detection performance. On real-world projects/modules, many specialized LLM-based and traditional methods exhibit high false discovery rates, while general-purpose agents reduce warning volume but remain susceptible to false positives. Based on Codex-assisted analysis of 6,442 sampled warnings with stratified manual validation, manual examination of 355 reports, and further classification of 5,896 false positives, we construct a taxonomy that identifies incorrect program-point classification, truncated interprocedural context, and semantic misunderstanding as prominent failure patterns. Our measurements of token consumption, monetary cost, and runtime further demonstrate that scalability remains a major obstacle, particularly for end-to-end LLM workflows. Finally, we distill these findings into six implications for developing more accurate, reliable, and cost-effective project-scale vulnerability detection methods.

REFERENCES

[1] B. Chess and G. McGraw, "Static analysis for security," *IEEE security & privacy*, vol. 2, no. 6, pp. 76–79, 2004.

[2] V. J. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2312–2331, 2019.

[3] P. Li, S. Yao, J. S. Korich, C. Luo, J. Yu, Y. Cao, and J. Yang, "Automated static vulnerability detection via a holistic neuro-symbolic approach," *arXiv preprint arXiv:2504.16057*, 2025.

[4] SpotBugs Team, "Spotbugs: Find bugs in java programs," https://spotbugs.github.io/, accessed: 2025-12-10.

[5] GitHub, "Codeql," https://codeql.github.com/, accessed: 2025-10-13.

[6] The Semgrep platform, "Semgrep," https://semgrep.dev/, accessed: 2025-10-13.

[7] Amazon Web Services, "Amazon codeguru," https://aws.amazon.com/codeguru/, accessed: 2025-12-10.

[8] SonarSource, "Sonarqube," https://www.sonarqube.org/, accessed: 2025-12-10.

[9] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *Proceedings of the 2014 IEEE Symposium on Security and Privacy (S&P)*, 2014. [Online]. Available: https://ieeexplore.ieee.org/document/6956589

[10] Clang and LLVM, "Clang static analyzer," https://clang-analyzer.llvm.org/, accessed: 2025-10-12.

[11] V. B. Livshits and M. S. Lam, "Finding security vulnerabilities in java applications with static analysis." in *USENIX security symposium*, vol. 14, 2005, pp. 18–18.

[12] J. Smith, B. Johnson, E. Murphy-Hill, B. Chu, and H. R. Lipford, "Questions developers ask while diagnosing potential security vulnerabilities with static analysis," in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 248–259.

[13] Z. Li, S. Dutta, and M. Naik, "Llm-assisted static analysis for detecting security vulnerabilities," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://arxiv.org/abs/2405.17238

[14] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, X. Wu, C. Tao, T. Zhang, and W. Liu, "Learning to detect memory-related vulnerabilities," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 2, pp. 1–35, 2023.

[15] W. Charoenwet, P. Thongtanunam, V.-T. Pham, and C. Treude, "An empirical study of static analysis tools for secure code review," in *Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis*, 2024, pp. 691–703.

[16] G. Lu, X. Ju, X. Chen, W. Pei, and Z. Cai, "Grace: Empowering llm-based software vulnerability detection with graph structure and in-context learning," *Journal of Systems and Software*, vol. 212, p. 112031, 2024.

[17] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[18] C. Zhang, H. Liu, J. Zeng, K. Yang, Y. Li, and H. Li, "Prompt-enhanced software vulnerability detection using chatgpt," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 2024, pp. 276–277.

[19] Y. Jiang, Y. Zhang, X. Su, C. Treude, and T. Wang, "Stagedvulbert: Multi-granular vulnerability detection with a novel pre-trained code model," *IEEE Transactions on Software Engineering*, 2024.

[20] S. Halder, M. E. Ahmed, and S. Camtepe, "Funcvul: An effective function level vulnerability detection model using llm and code chunk," in *European Symposium on Research in Computer Security*. Springer, 2025, pp. 166–185.

[21] P. Liu, J. Liu, L. Fu, K. Lu, Y. Xia, X. Zhang, W. Chen, H. Weng, S. Ji, and W. Wang, "Exploring {ChatGPT's} capabilities on vulnerability management," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 811–828.

[22] J. Guo, C. Wang, X. Xu, Z. Su, and X. Zhang, "Repoaudit: An autonomous LLM-agent for repository-level code auditing," in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: https://openreview.net/forum?id=TXcifVbFpG

[23] C. Wang, W. Zhang, Z. Su, X. Xu, X. Xie, and X. Zhang, "Llmdfa: analyzing dataflow in code with large language models," *Advances in Neural Information Processing Systems*, vol. 37, pp. 131 545–131 574, 2024.

[24] C. Yang, Z. Zhao, Z. Xie, H. Li, and L. Zhang, "Knighter: Transforming static analysis with llm-synthesized checkers," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, 2025, pp. 655–669.

[25] A. Lekssays, H. Mouhcine, K. Tran, T. Yu, and I. Khalil, "{LLMxCPG}:{Context-Aware} vulnerability detection through code property {Graph-Guided} large language models," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 489–507.

[26] C. Wang, J. Liu, X. Peng, Y. Liu, and Y. Lou, "Boosting static resource leak detection via llm-based resource-oriented intention inference," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2025, pp. 668–668.

[27] X.-C. Wen, X. Wang, Y. Chen, R. Hu, D. Lo, and C. Gao, "Vuleval: Towards repository-level evaluation of software vulnerability detection," *arXiv preprint arXiv:2404.15596*, 2024.

[28] M. D. Purba, A. Ghosh, B. J. Radford, and B. Chu, "Software vulnerability detection using large language models," in *2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW)*. IEEE, 2023, pp. 112–119.

[29] J. Lin and D. Mohaisen, "From large to mammoth: A comparative evaluation of large language models in vulnerability detection," in *Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS)*, 2025.

[30] A. Yildiz, S. G. Teo, Y. Lou, Y. Feng, C. Wang, and D. M. Divakaran, "Benchmarking llms and llm-based agents in practical vulnerability detection for code repositories," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 30 848–30 865.

[31] C. Thapa, S. I. Jang, M. E. Ahmed, S. Camtepe, J. Pieprzyk, and S. Nepal, "Transformer-based language models for software vulnerability detection," in *Proceedings of the 38th annual computer security applications conference*, 2022, pp. 481–496.

[32] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *2024 IEEE symposium on security and privacy (SP)*. IEEE, 2024, pp. 862–880.

[33] Z. Gao, H. Wang, Y. Zhou, W. Zhu, and C. Zhang, "How far have we gone in vulnerability detection using large language models," *arXiv preprint arXiv:2311.12420*, 2023.

[34] B. Steenhoek, M. M. Rahman, M. K. Roy, M. S. Alam, E. T. Barr, and W. Le, "A comprehensive study of the capabilities of large language models for vulnerability detection," *CoRR*, 2024.

[35] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the effectiveness of large language models in detecting security vulnerabilities," in *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 103–114.

[36] OpenAI, J. Achiam, S. Adler *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023. [Online]. Available: https://arxiv.org/abs/2303.08774

[37] Anthropic, "The claude 3 model family: Opus, sonnet, haiku," 2024, technical Report. [Online]. Available: https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf

[38] R. Anil, S. Borgeaud, J.-B. Alayrac *et al.*, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023. [Online]. Available: https://arxiv.org/abs/2312.11805

[39] T. Ahmed, K. S. Pai, P. Devanbu, and E. T. Barr, "Automatic semantic augmentation of language model prompts (for code summarization)," in *Proceedings of the 46th International Conference on Software Engineering (ICSE)*, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3597503.3639183

[40] Y. Dong, X. Jiang, J. Qian, T. Wang, K. Zhang, Z. Jin, and G. Li, "A survey on code generation with llm-based agents," *arXiv preprint arXiv:2508.00083*, 2025.

[41] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, and D. Phung, "Vulrepair: a t5-based automated software vulnerability repair," in *Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering*, 2022, pp. 935–947.

[42] T. Brown, B. Mann, N. Ryder, M. Subbiah *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: https://dl.acm.org/doi/abs/10.5555/3495724.3495883

[43] J. Wei, X. Wang, D. Schuurmans *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf
[44] Q. Wu, G. Bansal, J. Zhang, Y. Wu *et al.*, "Autogen: Enabling next-gen llm applications via multi-agent conversation," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://openreview.net/forum?id=tEAF9LBdgu
[45] GrammaTech, "Codesonar," https://www.grammatech.com/codesonar-cc, accessed: 2025-12-10.
[46] Facebook, "Infer static analyzer," https://github.com/facebook/infer, accessed: 2025-10-13.
[47] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.
[48] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2021, pp. 292–303.
[49] M. Fu and C. Tantithamthavorn, "Linevul: A transformer-based line-level vulnerability prediction," in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 608–620.
[50] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, and Z. Deng, "Sysevr: A framework for using deep learning to detect software vulnerabilities," *IEEE Transactions on Dependable and Secure Computing (TDSC)*, vol. 19, no. 4, pp. 2244–2258, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9321538
[51] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "Deepwukong: Statically detecting software vulnerabilities using deep graph convolutional neural networks," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 3, pp. 1–33, 2021. [Online]. Available: https://dl.acm.org/doi/10.1145/3436877
[52] X. Du, G. Zheng, K. Wang, Y. Zou, Y. Wang, W. Deng, J. Feng, M. Liu, B. Chen, X. Peng *et al.*, "Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag," *ACM Transactions on Software Engineering and Methodology*, 2024.
[53] Y. Yang, X. Zhou, R. Mao, J. Xu, L. Yang, Y. Zhang, H. Shen, and H. Zhang, "Dlap: A deep learning augmented large language model prompting framework for software vulnerability detection," *Journal of Systems and Software*, vol. 219, p. 112234, 2025.
[54] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 121–133.
[55] N. Risse, J. Liu, and M. Böhme, "Top score on the wrong exam: On benchmarking in machine learning for vulnerability detection," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 388–410, 2025.
[56] X. Wang, R. Hu, C. Gao, X.-C. Wen, Y. Chen, and Q. Liao, "Reposvul: A repository-level high-quality vulnerability dataset," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 2024, pp. 472–483.
[57] T. Liu, W. Ji, X. Dong, W. Yao, Y. Wang, H. Liu, H. Peng, and Y. Wang, "Jleaks: A featured resource leak repository collected from hundreds of open-source java projects," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
[58] OpenAI, "Codex," https://developers.openai.com/codex, accessed: 2026-05-08.
[59] Anthropic, "Claude code," https://claude.com/product/claude-code, accessed: 2026-05-08.
[60] L. De Moura and N. Bjørner, "Z3: An efficient smt solver," in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2008, pp. 337–340.
[61] P. Arteau, D. Formánek, and T. Polešovský, "OWASP Find Security Bugs," https://find-sec-bugs.github.io/, spotBugs plugin for security audits of Java web applications. Accessed: 2026-05-08.
[62] OpenAI, "Gpt-5.4," https://platform.openai.com/docs/models/gpt-5.4, accessed: 2026-05-08.
[63] Anthropic, "Claude sonnet 4.6," https://platform.claude.com/docs/en/models/sonnet-4-6/overview, accessed: 2026-05-08.
[64] OpenAI, "O3-mini," https://platform.openai.com/docs/models/o3-mini, accessed: 2025-10-13.
[65] ——, "Gpt-4," https://platform.openai.com/docs/models/gpt-4, accessed: 2025-10-13.
[66] Anthropic, "Claude sonnet 3.5," https://platform.claude.com/docs/en/about-claude/models/overview, accessed: 2025-10-13.
[67] Homepage, 2025, available at: https://github.com/Feng-Jay/LLM4Security.
[68] W. G. Cochran and W. G. Cochran, *Sampling techniques*. wiley New York, 1977, vol. 3, no. 4.
[69] N. K. Denzin and Y. S. Lincoln, *The Sage handbook of qualitative research*. sage, 2011.
[70] DeepSeek, "Deepseek v4 flash," https://api-docs.deepseek.com/quick_start/pricing/, accessed: 2025-10-13.
[71] X. Du, K. Yu, C. Wang, Y. Zou, W. Deng, Z. Ou, X. Peng, L. Zhang, and Y. Lou, "Minimizing false positives in static bug detection via llm-enhanced path feasibility analysis," *arXiv preprint arXiv:2506.10322*, 2025.
[72] H. Li, H. Zhang, K. Pei, and Z. Qian, "Towards more accurate static analysis for taint-style bug detection in linux kernel," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025, pp. 380–392.
[73] M. Lin and H. Chen, "Specauditor: Generating audit specifications for llm-driven bug detection," in *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2026, pp. 3396–3413.
[74] M. Maaz, L. DeVoe, Z. Hatfield-Dodds, and N. Carlini, "Agentic property-based testing: Finding bugs across the python ecosystem," *arXiv preprint arXiv:2510.09907*, 2025.
[75] N. Dhulshette, S. Shah, and V. Kulkarni, "Hierarchical repository-level code summarization for business applications using local llms," in *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*. IEEE, 2025, pp. 145–152.
[76] I. Bouzenia, P. Devanbu, and M. Pradel, "Repairagent: An autonomous, llm-based agent for program repair," *arXiv preprint arXiv:2403.17134*, 2024.
[77] R. Hong, H. Zhang, X. Pang, D. Yu, and C. Zhang, "A closer look at the self-verification abilities of large language models in logical reasoning," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 900–925.